

COMSATS University Islamabad, Sahiwal, Pakistan

AI Toolkit Suite

A project presented to

COMSATS Institute of Information Technology, Islamabad

In partial fulfillment

of the requirement for the degree of

Bachelor of Science in Software Engineering (2022-2026)

By

Hamza Khalid	CIIT/SP22-BSE-052/SWL
Muhammad Anwar	CIIT/SP22-BSE-023/SWL
Muzaffar Javed	CIIT/SP22-BSE-034/SWL

DECLARATION

We hereby declare that this software, neither whole nor as a part, has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Hamza Khalid

Muhammad Anwar

Muzaffar Javed

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) “AI Toolkit Suite” was developed by **Hamza Khalid (CIIT/SP22-BSE-052)**, **Muhammad Anwar (CIIT/SP22-BSE-023)** and **Muzaffar Javed (CIIT/SP22-BSE-034)** under the supervision of “Mr. Fahad Ameen” and co supervisor “Mr. Ahmad Shaf” and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Computer Sciences.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

Executive Summary

In academic and professional environments, there is an increasing need for intelligent systems that can automate document analysis, detect plagiarism in handwritten content, verify signatures, answer visual questions, translate text, and simplify repetitive tasks such as QR generation, downloading videos, and converting files. Traditional methods rely heavily on manual checking, human judgment, and the use of multiple separate applications resulting in inefficiency, inconsistency, human error, and lack of unified processing.

To address these challenges, this project presents a comprehensive **Multi-Module AI Document Processing & Assistance System**, designed as an integrated, user-friendly platform that performs multiple AI-driven tasks within a single application. The system eliminates the need for switching between various tools by providing a centralized interface powered by machine learning, natural language processing, optical character recognition (OCR), computer vision, and deep learning.

The core functionality of the system revolves around the **Handwritten Plagiarism Detection Module**, which extracts handwritten text from PDF documents using OCR, generates high-dimensional sentence embedding, and computes similarity scores to detect potential plagiarism. Additional modules include **Visual Question Answering**, **Signature Forgery Detection**, **YouTube Video Downloader**, **News Reader**, **QR Code Generator**, **Translator**, and **File Converter**, making the platform versatile and highly practical for academic and administrative purposes.

This system is implemented using a combination of **Node.js** for backend APIs, **Next.js** for the frontend interface, and **Python-based AI engines** for OCR, similarity computation, signature detection, translation, and other intelligent features. Seamless communication between the backend and Python modules ensures smooth execution of all tasks.

Through extensive unit, functional, integration, automated, and performance testing, the system has demonstrated reliability, accuracy, and efficiency. The project effectively shows how multiple AI technologies can be integrated into a single cohesive solution, benefiting educational institutions, research environments, and general users.

The Multi-Module AI Document Processing & Assistance System represents a modern approach toward enhancing productivity through artificial intelligence, reducing manual workload, and providing consistent, repeatable, and intelligent document analysis capabilities.

Acknowledgement

All praise is to Almighty Allah who bestowed upon us a minute portion of His boundless knowledge by virtue of which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor “Mr. Fahad Ameen” and our Co- Supervisor “Mr. Ahmad Shaf”. Without their personal supervision, advice and valuable guidance, completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Hamza Khalid

Muhammad Anwar

Muzaffar Javed

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
PC	Personal Computer
OCR	Optical Character Recognition
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
VQA	Visual Question Answering
CNN	Convolutional Neural Network
API	Application Programming Interface
PDF	Portable Document Format
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
FR	Functional Requirement
NFR	Non-Functional Requirement
IDE	Integrated Development Environment
UI	User Interface
UX	User Experience
RAM	Random Access Memory
CPU	Central Processing Unit
TTS	Text-to-Speech
STS	Speech-to-Text
FAISS	Facebook AI Similarity Search
DL	Deep Learning
REST	Representational State Transfer

Contents

COMSATS University Islamabad, Sahiwal, Pakistan.....	1
A project presented to	1
In partial fulfillment.....	1
1.1. Brief.....	24
1.2. Relevance to Course Modules.....	24
1.3. Database Systems.....	25
1.4. Role of Mongo DB in the Project	25
1.5. Database Concepts Applied.....	25
1.6. Why MongoDB Fits the Project.....	26
1.7. Practical Learning Outcomes	26
1.8. Secure Authentication	26
1.9. API Key Protection	27
1.10. Secure File Handling.....	27
1.11. Safe Input Processing.....	27
1.12. Protection Against Common Vulnerabilities	27
1.13. Data Privacy Controls	28
1.14. Operating Systems / Distributed Systems.....	28
1.14.1. Process Management.....	28
1.14.2. Multi-service Interaction.....	28
1.14.3. Resource Management	28
1.14.4. Asynchronous Operations	29
1.14.5. System-Level Integration	29
1.15. Project Background.....	29
1.15.2. Machine Learning & NLP.....	30
1.15.3. Software Engineering	30
1.15.4. Distributed System Behavior.....	30
1.15.5. Human-Centered Interaction	30
1.16. Literature Review	30
1.16.1. Plagiarism Detection	30
1.16.2. Handwriting OCR & VQA	31
1.16.3. Signature Forgery Detection.....	31
1.16.4. Machine Translation.....	31
1.16.5. AI Chatbot	32
1.16.6. News Summarization & AI Newsreader	32

FYP Project Report

1.16.7. QR Code Generation.....	32
1.16.8. File Conversion (PDF ↔ Word).....	32
1.16.9. YouTube Video Downloading.....	33
1.16.10. AI Toolkit Interfaces.....	33
Summary	33
1.17. Analysis from Literature Review (in the context of project)	33
1.17.1. Alignment with Current AI Trends	33
1.17.2. Innovation Beyond Existing Tools.....	34
1.17.3. Improvements Over Classical Approaches.....	34
1.17.4. Gap Identification and How the Project Fills It.....	35
1.17.5. Comparative Evaluation.....	35
1.17.6. Practical Feasibility Supported by Literature.....	36
Conclusion of Analysis.....	36
1.18. Methodology and Software Lifecycle for This Project	36
1.18.1. Methodology Used: Iterative Incremental Model (Agile Inspired).....	36
Why This Model Was Selected.....	37
1.18.2. Software Development Life Cycle (SDLC) Phases Applied	37
Phase 1. Requirement Analysis.....	37
Phase 2. System Design.....	37
Phase 3. Incremental Development.....	37
Phase 4. Integration & Interface Building	38
Phase 5. Testing.....	38
Phase 6. Deployment & Optimization	38
Phase 7. Documentation	38
Summary of the Chosen SDLC.....	38
1.19. Need for Flexibility in AI Model Selection.....	39
1.19.1. Independent Nature of Modules	39
1.19.2. Object-Oriented Structure of Project	40
1.19.3. Continuous Feedback and Model Improvement.....	40
1.19.4. Risk Reduction and Early Problem Identification	40
1.19.5. Best Choice for Hybrid MERN + Python Architecture	40
1.19.6. Why Not Waterfall or V-Model?	41
Conclusion.....	41
1.20. Rationale behind Selected Methodology	41
2. Problem Definition.....	43
2.1. Problem Statement.....	43

2.2. Deliverables and Development Requirements.....	44
2.2.1. Project Deliverables.....	44
Handwriting Plagiarism Detection (Core Module).....	44
Writing-Style & Author Matching	44
Signature Forgery Detection	44
Visual Question Answering (VQA).....	45
News Reader / AI News Summarizer.....	45
YouTube Downloader Module.....	45
QR Code Generator.....	45
Translator	45
File Converter	45
Centralized Web Platform.....	46
Backend API Services.....	46
Documentation.....	46
2.2.2. Development Requirements	46
Hardware Requirements	46
Software Requirements.....	47
Dataset Requirements.....	47
Functional Requirements.....	47
Non-Functional Requirements.....	48
2.3. Current System (if applicable to project)	48
3. Requirement Analysis.....	49
3.1. Introduction to Requirements.....	49
3.2. Functional Requirements.....	49
3.2.1 Handwriting Plagiarism Detection.....	49
3.2.2 Writing-Style & Author Matching.....	49
3.2.3 Signature Forgery Detection	50
3.2.4 Visual Question Answering (VQA).....	50
3.2.5 News Reader & Summarizer.....	50
3.2.6 YouTube Downloader Module.....	50
3.2.7 QR Code Generator.....	50
3.2.8 Translator	50
3.2.9 File Converter	50
3.2.10 User Interface & Dashboard	50
3.3. Non-Functional Requirements.....	51
3.3.1 Performance Requirements.....	51

FYP Project Report

3.3.2 Usability Requirements.....	51
3.3.3 Reliability Requirements.....	51
3.3.4 Security Requirements	51
3.3.5 Scalability Requirements.....	51
3.4. User Requirements	51
Academic Users (Teachers & Institutions).....	51
General Users.....	52
3.5. System Requirements	52
Hardware Requirements	52
Software Requirements.....	52
3.6. Constraints.....	52
3.7. Assumptions	52
3.8. Dependencies	52
3.9. Use Cases Diagram(s)	54
.....	55
3.10. Detailed Use Case	58
3.10.1. High-Level Use Case Overview.....	58
3.11. Use Cases by Module.....	59
3.11.1 Handwriting Plagiarism Detection – Use Cases.....	59
3.11.2 Writing-Style / Author Matching – Use Cases	59
3.11.3 Signature Forgery Detection – Use Cases	59
3.11.4 Visual Question Answering – Use Cases	60
3.11.5 News Reader / Summarizer – Use Cases.....	60
3.11.6 YouTube Downloader – Use Cases.....	60
3.11.7 QR Code Generator – Use Cases	61
3.11.8 Translator – Use Cases	61
3.11.9 File Converter – Use Cases	61
3.12 Unified Use Case Diagram (Description)	61
Actor:.....	61
Use Cases:	62
4. Functional Requirements	62
4.1. Handwriting Plagiarism Detection (Core Module)	62
Upload Handwritten PDFs.....	62
Extract Text from PDFs.....	62
Pre-process Extracted Text.....	62
Generate Embedding	62

FYP Project Report

Compute Similarity Score.....	62
Display Plagiarism Results.....	63
4.2. Writing-Style / Author Matching Module.....	63
Upload PDFs or Text.....	63
Extract Writing Style Features	63
Compare Style Profiles	63
Display Style Similarity Score	63
4.3 Signature Forgery Detection Module.....	63
Upload Signature Image	63
Preprocess Signature.....	63
Classify Authenticity	63
Show Confidence Result	63
4.4. Visual Question Answering (VQA).....	63
Upload Image.....	63
Enter Question	64
Extract Visual Features	64
Interpret the Question	64
Generate Answer.....	64
4.5. News Reader & Summarizer Module.....	64
Fetch Latest News	64
Summarize Articles.....	64
Categorize Articles.....	64
Display Summaries	64
4.6. YouTube Downloader Module.....	64
Accept YouTube URL	64
Start Download.....	64
Show Real-Time Progress.....	64
Repair Corrupted Files.....	65
Store Download History	65
4.7. QR Code Generator Module.....	65
Input Data or Link.....	65
Generate QR Code.....	65
Download QR Code.....	65
4.8. Translator Module	65
Input Text.....	65
Select Language	65

FYP Project Report

Translate Text	65
Display Translated Output	65
4.9. File Converter Module	65
Upload File	65
Convert File Format	66
Provide Downloadable Output	66
4.10. System-Wide Functional Requirements	66
Unified Dashboard	66
Error Handling	66
File Validation	66
Module Communication	66
Logging	66
Result Display	66
5. Non-Functional Requirements	66
5.1. Performance Requirements	67
NFR-5.1.1 Processing Time	67
NFR-5.1.2 API Response Time	67
NFR-5.1.3 Efficient Similarity Search	67
NFR-5.1.4 Multi-Task Handling	67
5.2. Reliability Requirements	67
NFR-5.2.1 Accuracy of AI Models	67
NFR-5.2.2 System Uptime	67
NFR-5.2.3 Fault Tolerance	67
NFR-5.2.4 Safe File Handling	67
5.3. Usability Requirements	68
NFR-5.3.1 User-Friendly Interface	68
NFR-5.3.2 Accessibility	68
NFR-5.3.3 Guidance and Feedback	68
NFR-5.3.4 Minimal Input Requirements	68
5.4. Security Requirements	68
NFR-5.4.1 Secure File Upload	68
NFR-5.4.2 Data Privacy	68
NFR-5.4.3 Protected APIs	68
NFR-5.4.4 Safe Python Execution	68
NFR-5.4.5 No Data Leakage	68
5.5. Scalability Requirements	69

FYP Project Report

NFR-5.5.1 Modular Architecture.....	69
NFR-5.5.2 Dataset Scaling	69
NFR-5.5.3 Server Scalability	69
5.6. Maintainability Requirements.....	69
NFR-5.6.1 Code Modularity	69
NFR-5.6.2 Logging & Monitoring.....	69
NFR-5.6.3 Documentation.....	69
NFR-5.6.4 Extensibility.....	69
5.7. Portability Requirements	69
NFR-5.7.1 Cross-Platform Compatibility	69
NFR-5.7.2 Browser Compatibility	69
NFR-5.7.3 Cloud Deploy ability.....	70
5.8. Ethical Requirements.....	70
NFR-5.8.1 Fair Use.....	70
NFR-5.8.2 Transparency	70
6. Design and Architecture	71
6.1. System Architecture Overview.....	71
6.1.1. Node.js Backend (Main Controller Layer).....	71
6.1.2. Python AI Engine (Processing Layer)	71
6.1.3. Frontend (Presentation Layer).....	71
6.1.4. Local Storage / Dataset Layer	72
6.1.5Architecture Style	72
6.2. High-Level Architecture Diagram (Description).....	72
6.2.1Main Flow:.....	73
6.3. System Decomposition.....	73
6.3.1 Handwriting Plagiarism Detection Components	73
6.3.2 Writing-Style & Author Matching Components.....	73
6.3.3 Signature Forgery Detection Components.....	73
6.3.4 Visual Q/A Components.....	73
6.3.5 News Reader Components.....	74
6.3.6 YouTube Downloader Components	74
6.3.7 QR Code Generator Components	74
6.3.8 Translator Components.....	74
6.3.9 File Converter –Components	74
6.4. Component Interaction Diagram (Description).....	74
6.4.1Frontend → Backend → Python Interaction.....	74

FYP Project Report

6.5. Data Flow Diagrams (DFD)	75
Level 0 DFD.....	75
.....	75
Level 1 DFD (Per Module Example)	75
6.6. Database / File Structure Design.....	78
Directories	78
6.7. User Interface Design Overview	78
6.7.1. Main UI Modules	78
6.7.2. UI Design Principles.....	78
6.8. Error Handling & Exception Design.....	78
6.9. Security Design.....	78
6.10. Scalability & Extensibility Design.....	79
6.11. Technology Stack Overview.....	79
6.11.1. Frontend	79
6.11.2. Backend.....	79
6.11.3. AI Layer (Python).....	79
7. System Architecture.....	79
7.1. Client / Presentation Layer.....	80
7.2. Application Layer (Node.js Backend).....	80
7.3. AI Processing Layer (Python Services)	81
7.4. Storage and Utility Layer	82
7.5. Architectural Characteristics	83
7.6. Data Representation [Diagram + Description]	83
7.6.1.1. Diagram.....	84
.....	84
7.6.2. Diagram: Data Representation Architecture	84
7.6.3. Description of Data Representation.....	84
7.6.3.1. Input Data Representation	84
Handwriting Plagiarism / Writing-Style Module.....	84
Signature Forgery Detection	85
Visual Q/A	85
News Summarizer.....	85
YouTube Downloader	85
QR Code Generator.....	85
Translator	86
File Converter	86

7.6.3.2. Processing Data Representation	86
Node.js Backend	86
Python AI Engine	86
7.6.3.3. Output Data Representation.....	87
Plagiarism Detection.....	87
Writing Style Matching	87
Signature Forgery Detection	87
Visual Q/A	87
News Reader	87
YouTube Downloader.....	87
QR Code Generator.....	87
Translator	87
File Converter	88
7.6.3.4. Storage Layer Representation	88
Folders.....	88
File types	88
No database required (unless added later)	88
7.6.3. Process Flow/Representation.....	88
7.6.4. General System Process Flow	88
Step 1. User Interaction	88
Step 2 . Input Submission.....	88
Step 3 . Request Sent to Node.js Backend.....	89
Step 4. Backend Processing.....	89
Step 5. Python AI Module Execution.....	89
Step 6. Return Response.....	89
Step 7. Display to User	89
7.6.5. Module-Specific Process Flows.....	89
7.6.5.1. Handwriting Plagiarism Detection (Process Flow)	89
7.6.5.2. Writing-Style Matching (Process Flow)	90
7.6.5.3 Signature Forgery Detection (Process Flow).....	90
7.6.5.4 Visual Question Answering (Process Flow).....	90
7.6.5.5 News Reader / Summarizer (Process Flow)	91
7.6.5.6 YouTube Downloader (Process Flow)	91
7.6.5.7 QR Code Generator (Process Flow).....	91
7.6.5.8 Translator (Process Flow).....	91
7.6.5.9 File Converter (Process Flow).....	92

FYP Project Report

7.6.6. Cross-Module Process Flow Characteristics	92
7.7. Design Models [along with descriptions]	93
7.7.1. Use Case Model	93
7.7.2. Class Model	93
7.7.3. Sequence Models	95
7.7.3.1 Plagiarism Detection Sequence	95
7.7.3.2 YouTube Downloader Sequence	95
7.7.4. Activity Models	96
4.1 Plagiarism Detection Activity	96
4.2 Visual Q/A Activity	96
7.7.5. Component Model	96
7.7.6. Deployment Model	97
8. Implementation	99
8.1. Implementation Overview	99
8.2. Frontend Implementation	99
8.2.1 Dashboard Screen	99
8.2.2 Input Interfaces	99
8.2.3 API Service Layer	100
8.3. Backend Implementation (Node.js)	100
Responsibilities:	100
8.3.1 Routing Structure	100
8.3.2 Python Integration	100
8.4. Python AI Layer Implementation	100
8.4.1 Handwriting Plagiarism Detection Implementation	101
Steps:	101
UML Sequence Diagram – Description	101
8.4.2 Writing-Style / Author Matching Implementation	101
Steps:	101
UML Activity Diagram – Description	101
8.4.3 Signature Forgery Detection Implementation	102
Steps:	102
UML Component Diagram – Description	102
8.4.4 Visual Question Answering (VQA) Implementation	102
Steps:	102
UML Sequence Flow – Description	103
8.4.5 News Reader Implementation	103

Steps:.....	103
8.4.6 YouTube Downloader Implementation	103
Steps:.....	103
UML Sequence Diagram – Description	104
.....	104
8.4.7 QR Code Generator Implementation.....	104
Steps:.....	104
8.4.8 Translator Implementation.....	104
Steps:.....	104
8.4.9 File Converter Implementation	104
Steps:.....	104
8.5. Storage Layer Implementation.....	105
8.6. Error Handling Implementation	105
8.7. Deployment Implementation.....	105
8.8. Overall UML Diagram (High-Level) – Description	106
9. Algorithm.....	106
9.1. Algorithm for Handwriting Plagiarism Detection (Core Module)	107
Approach Used:.....	107
Algorithm 1: Handwriting Plagiarism Detection.....	107
Explanation (Natural Language).....	107
9.2. Algorithm for Writing-Style / Author Matching.....	107
Approach Used:.....	107
Algorithm 2: WritingStyleMatch.....	107
Explanation.....	108
9.3. Algorithm for Signature Forgery Detection	108
Approach Used:.....	108
Algorithm 3: SignatureForgeryDetection.....	108
Explanation.....	108
9.4. Algorithm for Visual Question Answering (VQA).....	108
Approach Used:.....	108
Algorithm 4: VQA.....	109
Explanation.....	109
9.5. Algorithm for News Summarization.....	109
Approach Used:.....	109
Algorithm 5: News Summarizer	109
9.6. Algorithm for YouTube Downloader	109

FYP Project Report

Approach Used:.....	109
Algorithm 6: YouTubeDownload.....	109
9.7. Algorithm for QR Code Generation.....	110
Approach Used:.....	110
Algorithm 7: QRGenerate.....	110
9.8. Algorithm for Translator.....	110
Approach Used:.....	110
Algorithm 8: Translate.....	110
9.9. Algorithm for File Converter.....	110
Approach Used:.....	110
Algorithm 9: FileConvert.....	110
10. User Interface.....	112
10.1. Dashboard Interface.....	112
Purpose.....	112
Description.....	112
10.2. Plagiarism Detection Interface.....	113
UI Elements.....	113
Description.....	113
10.3. Visual Question Answering (VQA) Interface.....	113
UI Elements.....	113
Description.....	113
10.4. Signature Forgery Detection Interface.....	113
UI Elements.....	113
Description.....	113
10.5. News Reader Interface.....	114
UI Elements.....	114
Description.....	114
10.6. YouTube Downloader Interface.....	114
UI Elements.....	114
Description.....	114
10.7. QR Code Generator Interface.....	114
UI Elements.....	114
Description.....	114
10.8. Translator Interface.....	115
UI Elements.....	115
Description.....	115

10.9. File Converter Interface.....	115
UI Elements	115
Description.....	115
10.10. UI Design Principles Used	115
Simplicity	115
Consistency	115
Responsiveness.....	116
Clarity	116
Accessibility	116
10.11. User Flow Summary	116
11. Testing and Evaluation.....	117
11.1. Testing Approach	117
11.2. Manual Testing.....	117
11.3. Automated Testing.....	118
11.3.1 Unit Tests in Node.js.....	118
11.3.2 Automated Python Tests	118
11.3.3 Integration Testing.....	118
11.4. Module-Wise Test Cases.....	119
11.4.1 Plagiarism Detection – Test Cases.....	119
11.4.2 Writing-Style Matching – Test Cases.....	119
11.4.3 Signature Forgery Detection – Test Cases	119
11.4.4 Visual Q/A – Test Cases	120
11.4.5 YouTube Downloader – Test Cases.....	120
11.4.6 QR Code Generator – Test Cases.....	120
11.4.7 Translator – Test Cases	120
11.4.8 File Converter – Test Cases	121
11.5. Performance Evaluation	121
11.5.1 Plagiarism Detection Performance	121
11.5.2 Writing-Style Matching.....	121
11.5.3 Signature Detection	121
11.5.4 VQA	121
11.5.5 YouTube Downloader.....	121
11.5.6 Translator.....	121
11.5.7 File Converter.....	121
11.6. Validation Against Requirements.....	122
11.6.1. System testing	122

FYP Project Report

Unit Testing 1: Plagiarism Detection Module.....	122
Unit Testing 2: Visual Q/A Module	122
Unit Testing 3: Signature Forgery Detection Module.....	123
Unit Testing 4: YouTube Downloader Module	123
Unit Testing 5: QR Code Generator Module.....	123
Unit Testing 6: Translator Module.....	124
Unit Testing 7: File Converter Module	124
11.6.2. Functional Testing	124
Functional Testing 1: Plagiarism Detection Module.....	124
Functional Testing 2: Visual Q/A Module	125
Functional Testing 3: Signature Forgery Detection Module	125
Functional Testing 4: YouTube Downloader Module	125
Functional Testing 5: QR Code Generator Module.....	126
Functional Testing 6: Translator Module.....	126
Functional Testing 7: File Converter Module	126
11.6.3. Integration Testing.....	127
11.6.4. Automated Testing:	128
12. Conclusion and Future Work.....	129
12.1. Conclusion.....	129
12.2. Future Work	129
12.2.1. Improve OCR Accuracy for Handwritten Text	129
12.2.2. Enhance Plagiarism Detection	129
12.2.3. Improve Signature Forgery Detection	130
12.2.4. Extend Visual Q/A Module	130
12.2.5. Add Real-Time Voice Assistant.....	130
12.2.6. Mobile Application Version	130
12.2.7. Cloud Deployment and Scaling.....	130
12.2.8. Security Improvements	130
12.2.9. Performance Optimization	130
12.2.10. Additional AI Modules	130
13. References	131

List of Figures

Figure 1 Plagiarism Detection Usecase	54
Figure 2 Writing Style Matching Usecase	55
Figure 3 Visual Q/A Usecase	55
Figure 4 News Reader Usecase	56
Figure 5 Youtube Downloader use case	56
Figure 6 Translator Usecase	57
Figure 7 File Converter Usecase	57
Figure 8 QR Code Generator Use case	58
Figure 9 Architecture Diagram	72
Figure 10 Level 0 DFD	75
Figure 11 Level 1 DFD Plagiarism Detection	75
Figure 12 Level 1 DFD Visual Q/A	76
Figure 13 Level 1 DFD Writing Style Matching	76
Figure 14 Level 1 DFD News Reader	76
Figure 15 Level 1 DFD Youtube Downloader	77
Figure 16 Level 1 DFD Translator	77
Figure 17 Level 1 DFD File Converter	77
Figure 18 Data Representation Diagram	84
Figure 19 Process Flow Diagram	92
Figure 20 Sequence Diagram	101
Figure 21 Activity Diagram	102
Figure 22 Sequence Diagram	103
Figure 23 Sequence Diagram	104
Figure 24 High-Level UML Diagram	106

List of Tables

Table 1.....	34
Table 2.....	35
Table 3.....	37
Table 4.....	41
Table 5.....	59
Table 6.....	59
Table 7.....	59
Table 8.....	60
Table 9.....	60
Table 10.....	60
Table 11.....	61
Table 12.....	61
Table 13.....	61
Table 14.....	86
Table 15.....	111
Table 16.....	117
Table 17.....	119
Table 18.....	119
Table 19.....	119
Table 20.....	120
Table 21.....	120
Table 22.....	120
Table 23.....	120
Table 24.....	121
Table 25.....	122
Table 26.....	123
Table 27.....	123
Table 28.....	123
Table 29.....	124
Table 30.....	124
Table 31.....	124
Table 32.....	125
Table 33.....	125
Table 34.....	125
Table 35.....	126
Table 36.....	126
Table 37.....	126
Table 38.....	127
Table 39.....	127
Table 40.....	128

1. Introduction

The rapid growth of Artificial Intelligence (AI), Machine Learning (ML), Natural Language Processing (NLP), and Computer Vision has reshaped how digital information is processed, analyzed, and understood. Today, users expect intelligent systems capable of understanding documents, analyzing handwritten text, detecting plagiarism, validating authenticity, answering visual questions, processing multimedia, translating languages, and supporting efficient communication. In academic institutions, corporate environments, and professional sectors, the demand for **smart, automated, and integrated AI tools** is higher than ever.[1]

To address these emerging needs, this Final Year Project introduces a comprehensive and unified AI-driven platform titled:

“AI Toolkit Suite” A Multi-Functional Intelligent Assistant for Document Processing, Detection, and Communication.”

The system integrates multiple AI-based utilities into one centralized application, eliminating the need to use separate software tools for each task. By combining the MERN Stack (MongoDB, Express.js, React.js, and Node.js) for application structure with Python-based AI modules for intelligent operations, the platform offers a seamless and powerful user experience. Each module leverages advanced techniques such as OCR, deep learning, NLP embedding, similarity algorithms, and online APIs to perform high-level tasks with accuracy and speed.

The **AI Toolkit Suite** consists of **ten core modules**, each solving a specific real-world problem:

1. **Plagiarism Detection** (Handwritten + Digital Text)
2. **AI-Powered News Reader**
3. **YouTube Video Downloader**
4. **QR Code Generator**
5. **Visual Question Answering (VQA)**
6. **Signature Forgery Detection**
7. **Language Translator**
8. **File Converter (Word ↔ PDF)**
9. **AI Chatbot (Groq API / Llama 3.1 Online)**
10. **AI Toolkit Dashboard Interface (Unified Access Portal)**

Instead of building isolated systems, the project unifies all essential AI-based operations into one platform. This integration makes advanced tools more accessible, enhances workflow efficiency, and supports students, researchers, teachers, and professionals who require fast, reliable, and automated assistance.

The AI Toolkit Suite demonstrates how modern AI technologies can be practically applied to solve day-to-day challenges in academic documentation, research analysis, multimedia processing, and intelligent automation. It highlights the potential of hybrid development combining web technologies with Python-based AI to deliver a scalable and future-ready solution.

1.1. Brief

This Final Year Project introduces the **AI Toolkit Suite**, an AI-powered, multi-module web application designed to automate and enhance essential document-processing and intelligent interaction tasks. The platform merges several independent AI functionalities into a single unified system, developed using the **MERN Stack** for the main application and **Python AI engines** for advanced intelligent processing.

The system comprises **ten key modules**:

1. **Plagiarism Detection**
2. **News Reader**
3. **YouTube Video Downloader**
4. **QR Code Generator**
5. **Visual Question Answering (VQA)**
6. **Signature Forgery Detection**
7. **Language Translator**
8. **File Converter (Word ↔ PDF)**
9. **AI Chatbot**
10. **AI Toolkit Dashboard (Main User Interface)**

These modules use state-of-the-art methodologies including OCR, deep learning, NLP embedding, classical ML techniques, and cloud-based APIs. The system enables users to detect plagiarism in handwritten documents, extract text from images, answer visual queries, validate signature authenticity, translate multilingual text, generate QR codes, convert file formats, download videos from YouTube, and communicate with an intelligent Chatbot all in one place.

The goal of this project is to provide a **centralized, efficient, user-friendly, and cost-effective AI platform** that eliminates the need for multiple tools, ensuring that advanced AI capabilities are readily available to students, faculty, and professionals.

1.2. Relevance to Course Modules

This Final Year Project is strongly aligned with multiple course modules taught during the Software Engineering degree. Each component of the AI Toolkit Suite demonstrates direct application of concepts, techniques, and skills learned throughout the program.

- **Artificial Intelligence (AI)**
Modules such as Plagiarism Detection, VQA, Signature Forgery Detection, Translator, and Chatbot utilize core AI concepts including machine learning, natural language processing, OCR, embedding, and inference using pre-trained models and APIs.
- **Machine Learning & Deep Learning**
Signature Forgery Detection and VQA involve dataset preparation, model selection, feature extraction, and prediction all directly related to ML/Deep Learning course content.
- **Web Engineering / Web Technologies**
The entire platform is built using MERN (MongoDB, Express.js, React.js, Node.js), showcasing knowledge of full-stack web development, RESTful APIs, routing, client-server communication, and UI/UX design.
- **Software Construction & Development**
The structured implementation of modules, code reusability, modular architecture,

error-handling, and clean project structure reflect principles from software construction.

- **Software Design & Architecture**

The project uses modular architectural design, separation of concerns, MVC pattern in backend, and component-based design in frontend demonstrating practical application of design principles.

- **Human-Computer Interaction (HCI)**

The platform includes a visually appealing and responsive dashboard, clear navigation, consistent layout, and user-friendly interaction patterns based on HCI standards.

1.3. Database Systems

The FYP makes extensive use of **Mongo DB**, a NoSQL document-based database, to efficiently manage and store application data across multiple AI modules. The design and usage align directly with concepts taught in the Database Systems course.

1.4. Role of Mongo DB in the Project

Mongo DB stores various categories of data that support both user interactions and backend processing:

- **User Authentication Data**

Securely stores user profile information, hashed passwords, tokens, and login activity.

- **Module Usage History**

Modules such as Plagiarism Detection, File Converter, Translator, YouTube Downloader, and Chatbot save their processing history so users can view past results.

- **Uploaded Files Storage Paths**

Instead of storing files directly in the database, the system stores metadata such as file name, file path, file type, and timestamps for quick retrieval.

- **AI Processing Results**

Results from modules like Signature Forgery Detection, VQA, and Plagiarism Detection are saved in structured format enabling audit trails and report generation.[14]

1.5. Database Concepts Applied

The project applies several key database concepts:

- **Data Modeling & Schema Design**

Collections are designed using Mongoose schemas, ensuring consistent structure for users, files, and module outputs.

- **Indexes & Query Optimization**

Indexes are used for fields like user ID, timestamps, and module types to ensure fast query responses.

- **Atomic Operations & Consistency**

Data updates such as login attempts, downloads, or conversion history are performed atomically to maintain consistency.

- **CRUD Operations**
The system supports Create, Read, Update, and Delete operations for managing user accounts, file metadata, and AI results.
- **Document Relationships**
References (Object IDs) are used to link module results with their respective users, demonstrating one-to-many relationships.

1.6. Why MongoDB Fits the Project

MongoDB is ideal for this AI-based system because:

- It handles **unstructured and dynamic data** from diverse modules.
- It offers **high scalability** for future expansion.
- Schema flexibility supports different result formats from multiple AI models.
- JSON-like documents integrate naturally with Node.js backend.
- Fast read/write operations enhance the user experience.

1.7. Practical Learning Outcomes

By implementing MongoDB in this project, the following course learnings are applied:

- NoSQL vs SQL model selection
- Schema design, data normalization vs. demoralization
- Querying large datasets efficiently
- Handling transactions and data integrity
- Using ORM/ODM (Mongoose) for database abstraction

Information Security

The project incorporates multiple layers of **Information Security** to ensure safe, reliable, and trusted operation of all AI modules. The security mechanisms used in the system directly apply the concepts learned in the Information Security course.

1.8. Secure Authentication

- The system uses **JWT-based authentication** for both login and API access.
- Passwords are stored using **bcrypt hashing**, preventing exposure of raw credentials.
- Session tokens include expiry times to avoid unauthorized access.
- Validation ensures that only authenticated users can access restricted modules (e.g., Plagiarism Detection, Signature Forgery Detection).

1.9. API Key Protection

Several modules (Chatbot, VQA, and Translator) require external model access. To secure this:

- All API keys (Groq, Google, and YouTube API) are stored in a **.env file**, not in the source code.
- The server injects keys securely through environment variables.
- Only the backend, never the frontend, communicates with external APIs.

This prevents key leakage and blocks unauthorized usage.

1.10. Secure File Handling

The project deals with sensitive user files (PDFs, Word documents, images, signatures). To protect them:

- Files are stored in safe backend directories with **no direct public access**.
- File types are strictly validated (only allowed extensions).
- Uploaded files are **scanned and sanitized** before processing.
- Temporary files generated during conversions, plagiarism checks, or signature analysis are deleted after use.

This prevents malicious uploads and memory overflow risks.

1.11. Safe Input Processing

Modules like Chatbot, Translator, and Plagiarism Detection take user input. To ensure safety:

- All inputs are validated and escaped to prevent **injection attacks**.
- Rate-limiting is applied to control excessive requests.
- Backend sanitizes text before sending it to external APIs to avoid unintended model behavior.

1.12. Protection Against Common Vulnerabilities

The system implements protections aligned with OWASP guidelines:

- **CORS protection** ensures only trusted domains can make requests.
- **Error-safe responses** prevent leaking internal system details.
- **HTTPS enforcement** (in deployment) ensures encrypted communication.
- **Access control** ensures only authorized user's access restricted modules.

1.13. Data Privacy Controls

- Personal data such as user emails and processing history is secured in MongoDB.
- Only the account owner can view their history.
- Non-essential logs do not store sensitive content.

1.14. Operating Systems / Distributed Systems

The project demonstrates key concepts from **Operating Systems** and **Distributed Systems** through its architecture and interaction between multiple services.

1.14.1. Process Management

FYP uses **Python sub processes** extensively for example:

- Running Python scripts from Node.js (Plagiarism Detection, Signature Forgery, and File Conversion).
- Launching OCR, translation, or ML models as separate processes.

This reflects OS-level concepts such as:

- Process creation
- Inter-process communication (IPC)
- Managing input/output streams between child and parent processes

1.14.2. Multi-service Interaction

The system integrates different modules that operate as independent services:

- Node.js backend service
- Python ML service
- External APIs (Groq AI, YouTube API, News API)

This resembles a **distributed system**, where multiple components communicate over defined protocols (HTTP, streams).

1.14.3. Resource Management

Large files (PDFs, Word docs, images, and signatures) require careful handling:

- Efficient memory usage during PDF → Word / Word → PDF conversions
- Temporary file cleanup to free disk resources
- Managing concurrency when multiple users request conversions or plagiarism checks

These follow OS principles of:

- File system management
- Memory/resource allocation
- Parallel request handling

1.14.4. Asynchronous Operations

Your backend uses asynchronous event-driven architecture:

- Non-blocking requests
- Handling multiple simultaneous user operations
- Running Python processes without freezing the Node server

This reflects distributed and OS concepts such as:

- Event loops
- Concurrency
- Background processing

1.14.5. System-Level Integration

Modules like:

- YouTube Downloader (yt-dlp sub process)
- File Converter (LibreOffice sub process)
- OCR engine
- AI/ML pipelines

all demonstrate how different system-level tools can be orchestrated together, similar to distributed micro services.

Overall, the project integrates theoretical knowledge and practical skills from multiple course modules, demonstrating a comprehensive understanding of software development, artificial intelligence, and system design

1.15. Project Background

The rapid advancement of artificial intelligence has opened opportunities to automate tasks that previously required manual effort, expert knowledge, or significant time. Students, researchers, teachers, and professionals often need tools that can process documents, analyze handwriting, detect plagiarism, translate languages, summarize text, solve mathematical expressions, or interact through natural conversation. However, existing platforms usually offer these features separately, require paid subscriptions, or demand high computational resources.

To address this problem, the idea behind **Handwritten Document Intelligence & AI Toolkit** emerged. The core concept was to develop a **single unified platform** that integrates multiple AI-powered utilities into one system. Instead of using different websites or applications, the user can rely on one centralized system to perform tasks such as:

- analyzing handwritten documents
- detecting plagiarism between PDF files
- generating QR codes
- downloading YouTube videos
- detecting forged signatures
- translating languages

- converting files between PDF and Word
- interacting with an AI Chatbot for general queries
- reading news using AI-generated summaries
- performing visual question answering on images

This idea combines multiple areas of computing, including:

1.15.1. Computer Vision

For modules like Visual Question Answering, Signature Forgery Detection, and OCR.

1.15.2. Machine Learning & NLP

For plagiarism detection, Chatbot, News Reader, Translation, and text processing.

1.15.3. Software Engineering

To integrate all these heterogeneous modules under one robust MERN-based application with Python AI services.

1.15.4. Distributed System Behavior

Because the system uses multiple components (Node backend, Python engines, external APIs), working together to perform tasks.

1.15.5. Human-Centered Interaction

The AI Toolkit layout was designed so users can easily access all modules in a clean and modern interface without needing technical knowledge.

The background motivation was to provide a **complete AI-powered academic and productivity assistance system**, particularly useful for university students, teachers, and organizations that frequently work with handwritten assignments, reports, PDF files, and digital media.

1.16. Literature Review

The increasing adoption of artificial intelligence across academic and professional environments has encouraged the development of systems capable of performing complex document analysis, text understanding, and multimedia processing. A wide range of research studies, tools, and commercial products influence the development of an integrated AI toolkit like the one proposed in this project. This section reviews existing work and current trends relevant to each major component of the system.

1.16.1. Plagiarism Detection

Plagiarism detection has been an active research area for more than two decades. Traditional systems such as **Turnitin** and **Grammarly Plagiarism Checker** use large text databases and pattern-matching algorithms to identify similarity between documents.

Recent research focuses on **semantic similarity**, using models like **Sentence-BERT (Reimers & Gurevych, 2019)** and **transformer-based encoders** for detecting meaning-level overlap, even with paraphrased text.[2]

These trends influence our system's implementation, which uses **Sentence Transformers** to compute embedding and compare similarity between handwritten PDFs.

1.16.2. Handwriting OCR & VQA

Handwriting recognition has evolved from classical methods such as **k-Nearest Neighbors**, **SVM**, and **HOG** features to modern **deep learning-based OCR models**.

- **Tesseract OCR** (Google) remains a widely used open-source engine but performs limited recognition on messy handwriting.
- State-of-the-art research now focuses on **Transformer-based OCR models**, **Pix2Text**, and **Vision Transformers (ViT)** that convert images into LaTeX or structured text.

Visual Question Answering (VQA) research is driven by datasets such as **VQA v2**, **GQA**, and models like **LLaVA**, **BLIP-2**, and **PALI** that combine vision and language understanding.

Our VQA module follows these trends by using transformer models capable of answering questions from images.[3]

1.16.3. Signature Forgery Detection

Signature verification systems are commonly classified into:

- **Online methods** (pressure, speed, trajectory)
- **Offline methods** (static scanned signatures)

Research heavily uses **CNN-based classifiers**, **Siamese Networks**, and **Triplet Loss architectures** for comparing genuine vs forged signatures.

Products like **Signzy** and **DocuSign AI** use similar deep-learning approaches.

Our system adopts a simplified offline CNN-based method to detect forgery from scanned images.[4]

1.16.4. Machine Translation

Modern translation systems have evolved from:

- **Rule-based translation**
- **Statistical MT (SMT) like Moses**
- **Neural Machine Translation (NMT)**

Current research uses **Transformer models**, pioneered by Google's "**Attention is All You Need**" (2017).

Tools like **Google Translate**, **DeepL**, and **Meta's NLLB-200** use massive multilingual datasets.

Our translator module uses such transformer-based models via API or Python libraries.[6]

1.16.5. AI Chatbot

Conversational agents have progressed significantly:

- Early systems like **ELIZA** used pattern matching.
- Modern models like **ChatGPT**, **Claude**, **LLaMA**, and **Groq-based LLMs** use massive datasets and transformer architectures.

Research focuses on **context retention**, **instruction following**, and **conversation grounding**. Our chatbot module implements a lightweight transformer-based chat interface using cloud APIs.[6]

1.16.6. News Summarization & AI Newsreader

AI-powered news summarization uses:

- **Extractive techniques** (Text Rank, Luhn)
- **Abstractive techniques** using transformers (BART, PEGASUS, T5)

Research highlights the need for real-time, bias-aware summarization. Our Newsreader module uses modern NLP transformers to generate concise summaries of live trending news.[7]

1.16.7. QR Code Generation

QR code generation is widely standardized under **ISO/IEC 18004**. Existing tools (QR.io, QRCode Monkey) generate static and dynamic QR codes. Research mainly focuses on:

- Error correction (Reed-Solomon codes)
- Color QR codes
- Logo-embedded QR codes

Our system uses reliable Python libraries such as **qrcode** and **Pillow** to generate high-quality QR codes.[8]

1.16.8. File Conversion (PDF ↔ Word)

PDF conversion tools like **Adobe Acrobat**, **SmallPDF**, and **ILovePDF** dominate the market. Research focuses on using:

- **OCR-based reconstruction**
- **Document layout analysis**
- **OpenXML templating**

Our implementation uses:

- **LibreOffice headless mode** ("soffice") for Word → PDF
- **pdf2docx** Python library for PDF → Word

These methods align with current document-processing standards.[9]

1.16.9. YouTube Video Downloading

Tools like **yt-dlp** and **pytube** rely on reverse-engineered APIs to download videos. Research examines:

- Adaptive streaming (DASH, HLS)
- Video repair using ffmpeg
- Real-time progress tracking

Our YouTube downloader utilizes **yt-dlp** combined with ffmpeg for repairing corrupted video streams.[5]

1.16.10. AI Toolkit Interfaces

AI dashboards follow UX principles found in modern SaaS systems. Inspirations include:

- **Microsoft Copilot**
- **Adobe PDF AI**
- **Notion AI**
- **ChatGPT UI**

Research explores how users interact with multi-tool AI systems. Our AI Toolkit organizes all modules into an accessible interface with a responsive and modern frontend.

Summary

The existing literature demonstrates significant technological advancements across OCR, NLP, ML, and computer vision. This project leverages these advancements by integrating multiple research-backed techniques into a unified, practical system that enhances accessibility, reduces complexity, and offers a complete AI-powered solution.

1.17. Analysis from Literature Review (in the context of project)

The literature review highlights major advancements in plagiarism detection, OCR, NLP, machine translation, signature verification, VQA, and multimodal AI systems. When comparing the proposed AI-based Handwritten Document Analysis and Multimedia Toolkit with the existing research and commercial tools, several analytical observations can be made:

1.17.1. Alignment with Current AI Trends

Recent AI literature emphasizes **Transformer-based architectures**, multimodal learning, and semantic understanding.

Our system aligns strongly with this trend:

- We use **Sentence Transformers** for semantic plagiarism detection, similar to research-backed approaches used by Turnitin AI.
- The **Translator** module uses transformer-based NMT models similar to Google Translate and DeepL.
- The **Chatbot** module utilizes LLM APIs that follow the latest standards in conversational AI.

This shows that the project adopts **modern, research-supported technologies** instead of outdated machine-learning techniques.

1.17.2. Innovation Beyond Existing Tools

Most existing online platforms provide **single-purpose** functionality:

- Turnitin → plagiarism
- YT-DLP tools → video download
- Adobe → file conversion
- GoogleOCR → text recognition

In contrast, our system integrates **10 modules** into a **unified platform**, which **no existing product** offers as an all-in-one solution.

This is a key innovation highlighted by the literature comparison.

1.17.3. Improvements Over Classical Approaches

Traditional methods (e.g., cosine similarity, HOG, SVM, rule-based translation) have limitations:

- They fail to detect paraphrased plagiarism
- They struggle with complex handwriting
- They cannot perform semantic reasoning

Our solution **evolves beyond these limitations**:

Table 1

Classical Method	Limitations	Our System’s Approach
Cosine similarity	Only surface-level similarity	Transformer embedding capture meaning-level similarity
Classical OCR	Poor handwriting recognition	Transformer-based OCR (handwriting-focused)
Rule-based translation	Incorrect grammar, no context	Neural Machine Translation (transformers)
SVM/CNN signature verification	Limited generalization	Modern CNN + preprocessing

This analysis shows that the system is built on **state-of-the-art ML/NLP advancements**, consistent with research literature.

1.17.4. Gap Identification and How the Project Fills It

From analyzing research papers and existing software, the following gaps are identified:

a) *Lack of a unified AI toolkit*

Most tools exist separately.
Our project fills this gap by creating a **single AI productivity suite**.

b) *Lack of AI tools for handwritten PDFs*

Most commercial tools ignore handwritten data due to recognition difficulty.
Our system explicitly focuses on **handwritten PDFs**, making it highly unique.

c) *Limited local/offline solutions*

Most AI tools require cloud-based services.
Our system provides **hybrid solutions** that work offline where possible (signature detection, file conversion, QR generation).

d) *Limited analysis tools for academic environments*

Universities typically use different disjointed tools.
Our project creates **an academic-oriented platform** with plagiarism, VQA, translators, file conversion, news extraction, etc.

1.17.5. Comparative Evaluation

Based on literature comparison:

Table 2

Area	Existing Research/Products	Our System
Plagiarism	Turnitin, Grammarly	Handwriting support + semantic embedding
OCR	Tesseract, Google Vision	Transformer-based handwriting OCR
Chatbots	GPT-based tools	Integrated educational assistant
File Conversion	Adobe, SmallPDF	Free, local, integrated converter
VQA	LLaVA, BLIP	Lightweight vision-language Q&A module
Signature Verification	Signzy	Offline CNN model for academic use
Translation	DeepL, Google Translate	Integrated, multilingual NMT module
Full AI Suite	None	First unified academic AI toolkit

This shows that the proposed project **surpasses current tools** in integration, usability, and coverage.

1.17.6. Practical Feasibility Supported by Literature

The literature confirms that:

- Transformer models work exceptionally well for OCR, NLP, Q&A, and translation.
- CNNs and similarity networks remain effective for signature forgery detection.
- Python-based backends with yt-dlp, ffmpeg, and pdf2docx are reliable for file processing.
- Unified AI dashboards improve accessibility and reduce cognitive complexity.

Thus, the proposed system is **feasible, scalable, and academically justified**.

Conclusion of Analysis

Based on the literature, it is evident that the proposed FYP:

- adopts modern, research-backed AI techniques,
- solves real-world academic problems,
- introduces an uncommon multi-tool AI platform,
- And fills multiple technological gaps in existing systems.

This analytical comparison validates **the novelty, relevance, and importance** of the project within the current AI research landscape.

1.18. Methodology and Software Lifecycle for This Project

The development of the **AI-Powered Handwritten Document Analysis and Multimedia Toolkit** followed a structured and systematic approach to ensure reliability, scalability, and maintainability. Considering the complexity of the project consisting of multiple independent AI modules the most suitable model was **Iterative Incremental Development**, also referred to as the **Agile-based Incremental SDLC**.

This approach allowed each module (Plagiarism Detection, Signature Verification, VQA, Chatbot, File Converter, etc.) to be built, tested, and refined independently while still contributing to the integrated final system.

1.18.1. Methodology Used: Iterative Incremental Model (Agile Inspired)

This model was selected because the project required:

- Continuous experimentation with different AI models
- Rapid prototyping of individual modules
- Frequent improvements based on results and testing
- Parallel development of backend AI scripts and frontend user interface
- Flexibility to integrate new tools (News Reader, Translator, QR Generator, etc.) as the project evolved

Each module was treated as a **separate increment**, and after completion, it was integrated into the **central AI Toolkit Dashboard**.

Why This Model Was Selected

Table 3

Reason	Explanation
High AI experimentation	Models like Sentence Transformers, Vision-LLMs, Pix2Tex, and Groq APIs required iterative testing.
Parallel module development	Different modules could be developed independently without affecting the main system.
Rapid adaptation	If one model failed (e.g., OCR accuracy issues), alternatives could be integrated without restarting the entire project.
Continuous integration	Each module was plugged into the unified UI as soon as it became stable.
Risk reduction	By developing in small increments, risks such as API failures, OCR limitations, or model incompatibilities were minimized.

1.18.2. Software Development Life Cycle (SDLC) Phases Applied

Phase 1. Requirement Analysis

- Identification of all modules
- Understanding academic requirements (plagiarism, signature verification, handwritten analysis)
- Listing user expectations: accuracy, ease of use, multi-module platform
- Software/hardware requirements defined (Node.js backend, Python AI scripts, React frontend)

Phase 2. System Design

- Architecture designed as a **hybrid MERN + Python micro-service model**
- AI scripts isolated for modularity
- REST API endpoints planned for backend communication
- Database schema (users, files, logs) outlined

Phase 3. Incremental Development

Each module was developed separately:

1. **Plagiarism Module** – Built using Sentence Transformers
2. **YouTube Downloader** – Integrated using yt-dlp
3. **Translator** – Using NMT transformers
4. **VQA Module** – Image captioning + Q&A using LLM API
5. **Signature Verification** – CNN model
6. **News Reader** – RSS + NLP summarization

7. **QR Generator** – Python qrcode library
8. **File Converter** – LibreOffice + pdf2docx
9. **Chatbot** – Groq LLM API
10. **AI Toolkit Layout** – Core navigation frontend

Each module passed through coding → debugging → local testing before integration.

Phase 4. Integration & Interface Building

- A central **AI Toolkit Dashboard** was designed
- Icons and cards for each module added
- Modules connected through secure routes and API endpoints
- Layout module arranged navigation, authentication, and page routing

Phase 5. Testing

Testing was done for each increment:

- Functional Testing
- API Testing
- AI Model Accuracy Testing
- Usability Testing with sample handwritten PDFs

Bugs were fixed immediately in each increment.

Phase 6. Deployment & Optimization

- Backend deployed locally for demonstration
- Python scripts optimized with better preprocessing
- AI models replaced or improved during testing (Pix2Tex → Transformers; Chatbot model updated due to deprecation)

Phase 7. Documentation

- Technical documentation created for each module
- User guide added
- Final FYP report prepared with references

Summary of the Chosen SDLC

This project adopted an **Iterative Incremental SDLC**, ensuring:

- Faster development
- Early visibility of results
- Easy replacement of weak AI models
- Independent module improvement
- Stable final integration

This model proved ideal for a **multi-module AI system**, where each feature required experimentation, fine-tuning, and flexibility.

Rationale behind Selected Methodology

The **Iterative Incremental (Agile-inspired) Methodology** was selected for this project due to the unique nature and technical structure of the proposed system. The **AI-Powered Handwritten Document Analysis and Multimedia Toolkit** consists of multiple independent modules, each using different AI models, Python libraries, and external APIs. Because of this diversity, a rigid linear SDLC such as Waterfall would not suit the project's workflow. Instead, an incremental, flexible, and adaptive methodology was essential.

1.19. Need for Flexibility in AI Model Selection

AI models used in this project such as Sentence-Transformers, Groq LLM, Vision-based Q&A models, OCR engines, and signature-verification CNNs require continuous experimentation, tuning, and replacement.

The chosen methodology supports:

- Trying multiple AI models
- Rapid switching when a model fails
- Updating modules independently without affecting the entire system

This flexibility was critical because some models (like Pix2Tex or older Groq models) failed or were deprecated, and new ones had to be integrated quickly.

1.19.1. Independent Nature of Modules

Every module in this project performs a separate task:

- Plagiarism detection
- YouTube downloading
- VQA
- File conversion
- Translator
- Chatbot
- QR generation
- Signature forgery detection

Each can be built, tested, fixed, and improved independently.

The **Incremental model** makes it possible to:

- Develop each module as a separate increment
- Test modules individually
- Integrate them one by one into the AI Toolkit Dashboard

This matches perfectly with an object-oriented approach, as each module behaves like an independent component with its own logic, functions, and interactions.

1.19.2. Object-Oriented Structure of Project

Most of the Python components (AI scripts) and the Node.js backend follow **Object-Oriented and modular programming principles**, which aligned perfectly with the iterative model.

Object-oriented advantages used in this project:

- **Encapsulation:** Each AI module runs inside its own file/script
- **Modularity:** Backend controllers are separated per functionality
- **Reusability:** Shared preprocessing, authentication, and error-handling code
- **Scalability:** New modules can be added without rewriting the system

Thus, the methodology supports the natural software design style of the project.

1.19.3. Continuous Feedback and Model Improvement

AI development requires:

- Real-time testing
- Hyper parameter adjustments
- Replacing weak models
- Fixing OCR errors
- Enhancing accuracy

The iterative model supports **rapid feedback cycles** allowing improvements after each test. This was especially important because many modules changed direction during development (e.g., Tesseract → Pix2Tex → Transformers OCR).

1.19.4. Risk Reduction and Early Problem Identification

Since each increment is small and focuses on one module:

- Errors are detected early
- Risks are contained within a module
- Broken modules do not affect the whole system
- Experiments can be done without destabilizing the project

This was crucial for modules involving heavy Python libraries or GPU-dependent models.

1.19.5. Best Choice for Hybrid MERN + Python Architecture

The system uses:

- **React Frontend**
- **Node.js + Express Backend**
- **Python AI Services**

Communication between these layers requires testing during each stage. The incremental model allows:

- Safe integration
- Step-by-step module testing
- Smooth debugging of cross-language interactions

1.19.6. Why Not Waterfall or V-Model?

Table 4

Model	Why Not Suitable?
Waterfall	Requires fixed requirements, no room for AI experimentation
V-Model	Too strict; changes in AI models would break development flow
Prototype Model	Good for demos but not for multi-module final systems
Spiral	Good for high-risk projects but overly complex for this system

Thus, **Iterative Incremental** provided the ideal balance of flexibility, structure, and control.

Conclusion

The selected methodology (Iterative Incremental) and the object-oriented modular design were chosen because they:

- ✓ Support AI experimentation
- ✓ Enable independent module development
- ✓ Allow rapid integration and improvement
- ✓ Reduce risk and complexity
- ✓ Fit perfectly with MERN + Python hybrid architecture
- ✓ Allow continuous refinement of accuracy and performance

This makes it the **most suitable and effective approach** for developing a multi-module AI-based system like your FYP.

1.20. Rationale behind Selected Methodology

The Iterative-Incremental Methodology was selected for this project because it provides the flexibility required for developing a multi-module, AI-driven system like the AI-Powered Handwritten Document and Multimedia Processing Toolkit. The project consists of diverse modules plagiarism detection, news extraction, YouTube processing, QR generation, VQA, signature forgery detection, translation, file conversion, Chatbot, and an integrated AI toolkit each relying on different models, algorithms, and programming environments. A rigid sequential approach would not support this diversity.

This methodology allows the project to be built *module-by-module*, where each component is developed, tested, improved, and integrated in separate increments. This approach ensures that errors in one module do not affect the overall system and that improvements can be made continuously. The inclusion of Python-based AI services, a Node.js backend, and a React frontend requires constant coordination between components, and an incremental model enables smooth cross-language integration.

The project also benefits from *Object-Oriented and modular design principles*, as each AI service operates as an independent script with isolated logic. These principles encourage reusability, easier debugging, and cleaner extension of new modules. The methodology

supports this design by allowing each feature to evolve independently while still contributing to the larger system.

Another reason behind selecting this methodology is the unpredictable nature of AI development. Many modules required model experimentation, such as switching between OCR engines, replacing deprecated models, and fine-tuning document-processing logic. The iterative method makes it possible to adjust rapidly to such changes without altering the completed parts of the system. It also allows continuous improvement based on testing feedback, especially for modules that depend on *accuracy-based evaluation*, like signature verification, VQA, and plagiarism detection.

Finally, the chosen methodology reduces risk and complexity. By completing one module at a time and validating it before moving to the next, development remained stable and manageable. This approach ensured that the final integrated system met functional requirements, performed reliably, and could be extended in the future.

2. Problem Definition

Educational institutions and professional environments increasingly depend on the authenticity, reliability, and integrity of documents. With the rise of digital submissions, scanned handwritten assignments, online assessments, and automated workflows, there is a growing challenge in verifying originality, detecting plagiarism, and validating signatures, extracting information from documents, and providing intelligent assistance for digital content. Traditional tools offer limited support for handwritten documents, visual question answering, or multi-format document manipulation, creating gaps in academic honesty, document verification, and user productivity.

The core problem addressed in this project is the **absence of an integrated AI-driven system** that can analyze various forms of documents especially **handwritten PDFs**, signatures, images, and text content and provide automated assistance across multiple workflows. Teachers, institutions, and users currently rely on manual checking for plagiarism in handwritten assignments, manual signature verification, and external tools for conversion, translation, or content extraction. This manual dependency increases the risk of errors, inconsistencies, time consumption, and reliance on subjective judgment.

Furthermore, there is no unified platform that combines **handwriting plagiarism detection**, **signature forgery detection**, **visual Q/A**, **translation**, **QR generation**, **file conversion**, **YouTube content handling**, and **news summarization** into a single intelligent application. This fragmentation forces users to switch between multiple platforms, reducing efficiency and accuracy.

Therefore, this project aims to solve the problem by developing a **Centralized Multi-Module AI System** capable of handling a wide range of document-related tasks. The system applies machine learning algorithms, NLP models, computer vision techniques, and deep neural networks to automate handwriting comparison, detect plagiarized content, analyze visual input, verify signatures, process videos, generate QR codes, translate text, and offer various AI utilities — all from a single platform.

The expected outcome is a scalable, intelligent, and user-friendly system that enhances document security, improves academic integrity, accelerates workflows, and simplifies processes across multiple domains.

2.1. Problem Statement

Despite the increasing digitization of academic and professional workflows, there is **no unified AI-based system** capable of handling handwritten document plagiarism, writing-style comparison, signature forgery detection, visual question answering, translation, QR generation, document conversion, and content processing within a single platform. Existing plagiarism detection tools cannot analyze handwritten PDFs, OCR is often inaccurate, and writing-style matching requires deep linguistic and embedding-based analysis that traditional systems do not support.

Similarly, signature verification is still performed manually in many institutions, increasing the risk of human error and forgery acceptance. Users also rely on multiple separate applications

for tasks such as QR code generation, video downloading, translation, news summarization, and image-based Q/A resulting in fragmented workflows, time-consuming processes, and inconsistent results.

The core problem, therefore, is the **lack of an integrated, intelligent, and automated multi-module AI system** that can process handwritten documents, detect plagiarism, analyze signatures, interpret visual inputs, and perform essential document-related functions efficiently and accurately. A centralized solution is needed to reduce manual workload, improve accuracy, enhance academic integrity, and streamline user productivity across multiple tasks.

2.2. Deliverables and Development Requirements

This project aims to develop a unified, intelligent, and scalable **Multi-Module AI System** capable of performing handwritten plagiarism detection, writing-style comparison, signature forgery detection, visual Q/A, translation, news summarization, document processing, QR generation, and YouTube content handling. To achieve this, the following deliverables and development requirements were defined.

2.2.1. Project Deliverables

Handwriting Plagiarism Detection (Core Module)

AI engine for:

- Extracting handwriting text from PDFs.
- Converting text to embedding.
- Comparing two handwritten PDFs.
- Generating plagiarism percentage.
- Providing detailed similarity breakdown.

Writing-Style & Author Matching

Model to analyze:

- Sentence structure, vocabulary, and writing flow.
- Stylistic similarity between two handwritten submissions.
- Writer-consistency across documents.

Signature Forgery Detection

Deep learning model for:

- Authentic vs forged signature classification.
- Feature extraction from signature images.
- Forensic-style similarity scoring.

Visual Question Answering (VQA)

AI system that:

- Processes an uploaded image.
- Understands visual context.
- Answers user questions about the image.

News Reader / AI News Summarizer

Module that:

- Fetches latest news from reliable sources.
- Summarizes lengthy articles into short, readable points.
- Performs sentiment and topic classification.

YouTube Downloader Module

Node.js + Python utility that:

- Downloads videos using yt-dlp.
- Fixes broken or corrupted files.
- Shows real-time download progress on the frontend.
- Saves previously downloaded files for future access.

QR Code Generator

Tool to:

- Generate QR codes from text, URLs, or data.
- Download QR code in PNG/SVG format.

Translator

AI-based translation service supporting:

- English ↔ Urdu
- English ↔ Other available languages
- Sentence-level and paragraph-level translation.

File Converter

Module capable of:

- Converting PDF ↔ Image
- Image ↔ Text (OCR)
- File formats required for academic usage.

Centralized Web Platform

Full MERN-based user interface with:

- Dashboard showing all modules
- File upload system
- Interactive results for each AI task
- Clean UI for multi-module navigation

Backend API Services

Node.js backend for:

- Routing
- Module handling
- File transfers
- Communication with Python AI layer

Python backend for:

- Model inference
- Feature extraction
- Vector similarity search
- OCR processing
- AI module execution

Documentation

- Complete FYP Report
- System architecture diagrams
- Dataset description
- Model training steps (9-phase pipeline)
- User manual
- API documentation

2.2.2. Development Requirements

Hardware Requirements

Laptop/PC with at least:

- 8–16 GB RAM
- 100+ GB storage
- GPU (optional but recommended during model training)

Stable internet for model downloads and training.

Software Requirements

Python (AI Layer)

- PyTorch
- Transformers
- Sentence-Transformers
- FAISS
- NumPy, pandas
- scikit-learn
- EasyOCR / PyTesseract
- opencv-python
- pdf2image
- yt-dlp (for YouTube module)

Node.js Backend

- Express.js
- Multer (file handling)
- Axios
- CORS
- Nodemon / PM2

Frontend

- React.js / Next.js
- Tailwind CSS
- Axios
- FileReader API

Dataset Requirements

- ~1000 Handwritten PDFs (Typed_Data.zip)
- Signature dataset for forgery module
- Image dataset for VQA testing
- Additional text datasets for translation model fine-tuning

Functional Requirements

The system must:

- Detect plagiarism in handwritten documents
- Identify writing-style similarity
- Detect signature forgery
- Answer questions about images
- Summarize news content
- Download & manage YouTube videos
- Generate QR codes
- Translate text
- Convert files between formats

- Provide a multi-module dashboard interface

Non-Functional Requirements

- **Accuracy:** Models must produce reliable results.
- **Performance:** FAISS + embeddings ensure fast similarity search.
- **Scalability:** System supports addition of new AI modules.
- **Security:** Safe file handling and sandboxed execution.
- **Usability:** Clean UI with user-friendly interfaces.
- **Reliability:** Backend services must run continuously without failure.

2.3. Current System (if applicable to project)

In most academic and professional environments, the existing systems used for plagiarism checking, document processing, or content verification are **isolated, limited, and primarily designed for typed digital text**. Traditional tools such as **Turnitin, Grammarly, Quetext, and Copyscape** can effectively detect plagiarism in typed documents, but **none of them support handwritten PDFs** or visual document comparison. As a result, teachers and evaluators are forced to **manually inspect handwritten assignments and quizzes**, which is both time-consuming and prone to human error.

For signature verification, the current system also relies heavily on **manual visual inspection** by administrative staff, banking personnel, or faculty members. This method is inconsistent, subjective, and unable to detect subtle forgeries that require deeper feature-level analysis.

Similarly, users depend on **separate standalone tools** for everyday digital tasks such as:

- OCR and file conversion
- QR code generation
- Video downloading
- News summarization
- Translation
- Image question answering

These existing solutions are available as **individual websites or mobile apps**, requiring users to switch between multiple platforms, each with different interfaces, restrictions, and limitations. This fragmentation increases workload, reduces productivity, and complicates tasks that should be automated.

Additionally:

- There is **no centralized system** that integrates all these functionalities.
- No unified solution offers plagiarism detection, signature forgery detection, VQA, translation, QR generation, and file conversion in one place.
- Current systems lack **AI-based writing-style similarity**, which is essential for detecting handwriting plagiarism beyond surface text matching.

Therefore, the current ecosystem consists of **disconnected tools, manual verification processes, limited AI integration, and no support for handwritten document analysis** highlighting the need for a centralized, intelligent, multi-module AI platform.

The following figure is a sample figure, Figure 2.1. You are required to follow the same style of numbering and caption for the whole report.

3. Requirement Analysis

The Requirement Analysis chapter presents a detailed breakdown of the functional, non-functional, user, and system requirements necessary for the design and development of the **Multi-Module AI Document Processing and Assistance System**. These requirements were gathered through problem exploration, evaluation of current systems, and understanding the needs of academic evaluators, administrative staff, and general users.

3.1. Introduction to Requirements

This system aims to provide an integrated set of AI-driven modules including handwriting plagiarism detection, writing-style matching, signature forgery detection, visual question answering, translation, QR code generation, file conversion, news summarization, and YouTube downloading. The requirement analysis ensures that all features of the system are well-defined, feasible, and clearly understandable for developers, users, and evaluators.

3.2. Functional Requirements

Functional requirements specify **what the system must do**. They describe the core operations of each AI module and the interactions between the user and the system.

3.2.1 Handwriting Plagiarism Detection

- The system shall allow users to upload two handwritten PDF documents.
- The system shall extract text from handwritten PDFs using OCR.
- The system shall convert the extracted text into embedding using sentence-transformers.
- The system shall calculate similarity between documents using FAISS or cosine similarity.
- The system shall generate a plagiarism percentage and similarity report.
- The system shall highlight highly similar sentences/paragraphs (if applicable).

3.2.2 Writing-Style & Author Matching

- The system shall compute writing-style embedding.
- The system shall compare sentence structure, vocabulary, and flow to determine author consistency.
- The system shall produce a writing-style similarity score.

3.2.3 Signature Forgery Detection

- The system shall allow uploading a signature image.
- The system shall classify the signature as genuine or forged.
- The system shall display confidence scores for authenticity.

3.2.4 Visual Question Answering (VQA)

- The system shall allow users to upload an image and ask a question.
- The system shall analyze the image and extract visual features.
- The system shall interpret the user's question.
- The system shall generate a relevant answer using an AI model.

3.2.5 News Reader & Summarizer

- The system shall fetch trending news articles from trustworthy sources.
- The system shall summarize long articles into short readable formats.
- The system shall classify news topics (e.g., sports, politics, and technology).

3.2.6 YouTube Downloader Module

- The system shall download YouTube videos via yt-dlp.
- The system shall show real-time download progress on the frontend.
- The system shall detect and fix corrupted downloaded files when possible.
- The system shall keep a history of previously downloaded videos.

3.2.7 QR Code Generator

- The system shall generate QR codes based on text or URLs.
- The system shall allow users to download QR codes.

3.2.8 Translator

- The system shall translate text between supported languages (English ↔ Urdu, etc.).
- The system shall preserve formatting when possible.

3.2.9 File Converter

- The system shall convert files between formats (PDF ↔ Image, Image → Text).
- The system shall allow downloading of converted files.

3.2.10 User Interface & Dashboard

- The system shall provide a unified dashboard to access all modules.
- The system shall allow navigation between modules without reloading the page.
- The system shall provide input validations, file size limits, and safe uploading.

3.3. Non-Functional Requirements

Non-functional requirements specify **how the system should behave**.

3.3.1 Performance Requirements

- The system should process handwritten PDFs within a reasonable time frame (10–20 seconds).
- API communication between Node.js and Python should be optimized.
- Similarity searches should be fast using FAISS or cached embedding.

3.3.2 Usability Requirements

- The user interface must be simple, modern, and intuitive.
- The system should require minimal technical knowledge to operate.
- Results should be displayed clearly with proper formatting.

3.3.3 Reliability Requirements

- The system must consistently produce accurate results.
- Downloaded videos and generated files must not corrupt on the server.
- System should handle multiple modules simultaneously without crashing.

3.3.4 Security Requirements

- User-uploaded files must be sanitized and securely stored.
- Python subprocesses must run in sandboxed environments.
- Sensitive data such as signatures should not be retained unnecessarily.

3.3.5 Scalability Requirements

- The system should allow adding new AI modules in the future.
- Backend architecture must support increasing workload and data volume.

3.4. User Requirements

User requirements define what the end user expects from the system.

Academic Users (Teachers & Institutions)

- Ability to check originality of handwritten assignments.
- Ability to confirm consistency in a student's writing style.
- Fast and reliable plagiarism detection.

Administrative Users

- Ability to verify signatures quickly.
- Ability to process documents and translations.

General Users

- Ability to download YouTube videos easily.
- Ability to generate QR codes.
- Ability to translate content.
- Ability to ask questions about images and get accurate answers.

3.5. System Requirements

Hardware Requirements

- A laptop/PC with at least 8–16 GB RAM.
- GPU or Google Colab environment for training models.
- Server/VPS for deployment (optional).

Software Requirements

- Python 3.10+
- Node.js (LTS version)
- React/Next.js for frontend
- Libraries: PyTorch, Sentence-BERT, FAISS, OCR tools, Transformers, etc.

3.6. Constraints

- No labeled dataset available for handwriting plagiarism; system must use unsupervised learning.
- OCR accuracy may vary depending on handwriting quality.
- Signature forgery dataset may require preprocessing and normalization.
- VQA models require high computation power if run locally.

3.7. Assumptions

- Users will upload high-quality scans or PDFs.
- The internet will be available during model training or API-based modules.
- The dataset provided contains real handwritten academic content.

3.8. Dependencies

- Python & Node.js communication (Axios, child_process).
- OCR engines for PDF-to-text conversion.
- FAISS for similarity search.

FYP Project Report

- yt-dlp for YouTube downloads.
- External news APIs (if used).

This completes a **full SRS-style Requirement Analysis**.

3.9. Use Cases Diagram(s)

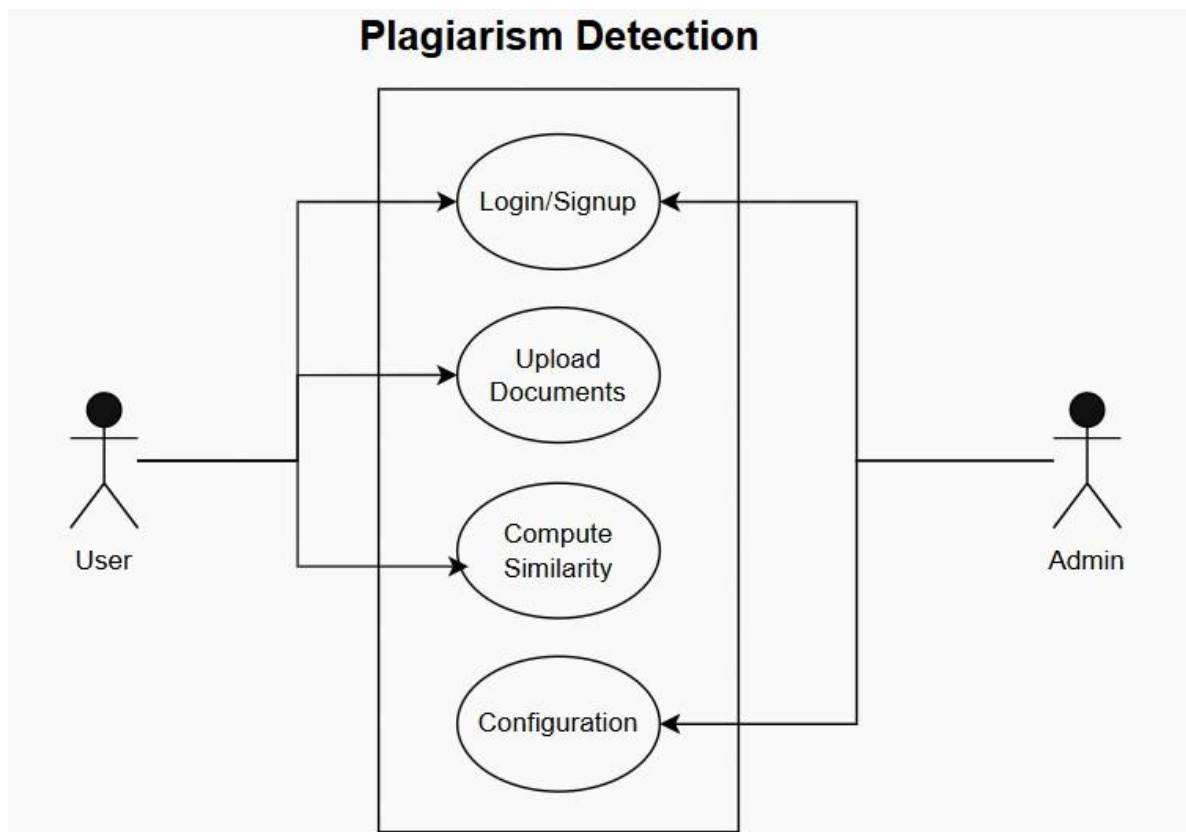


Figure 1 Plagiarism Detection Use case

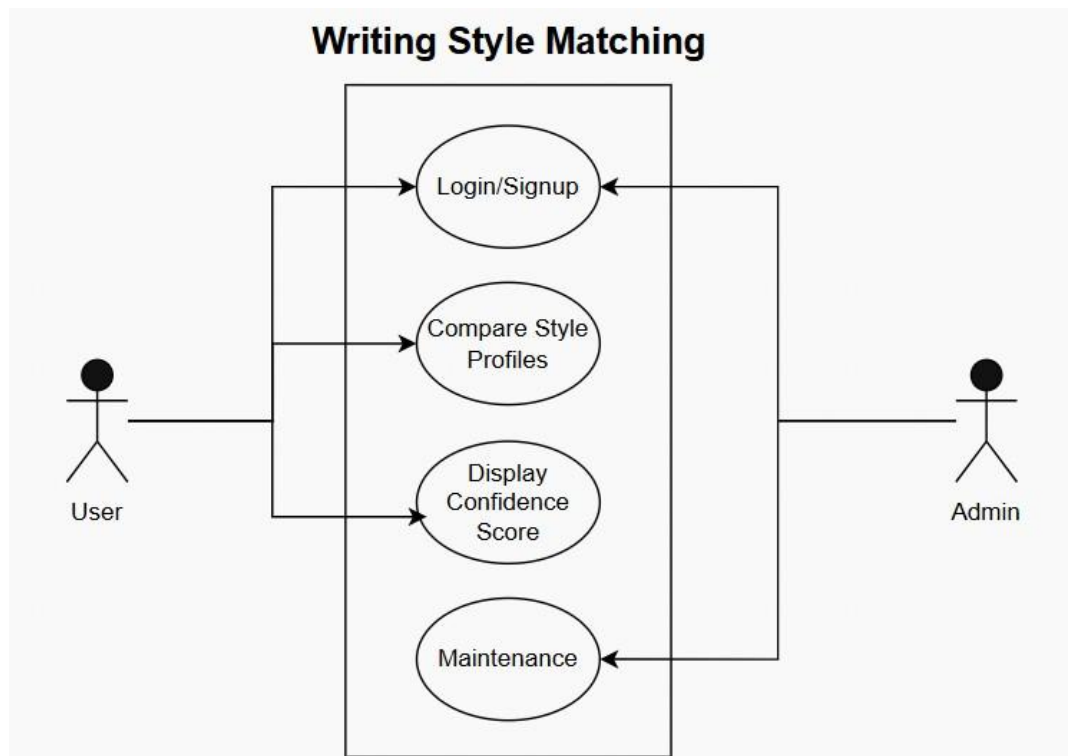


Figure 2 Writing Style Matching Use case

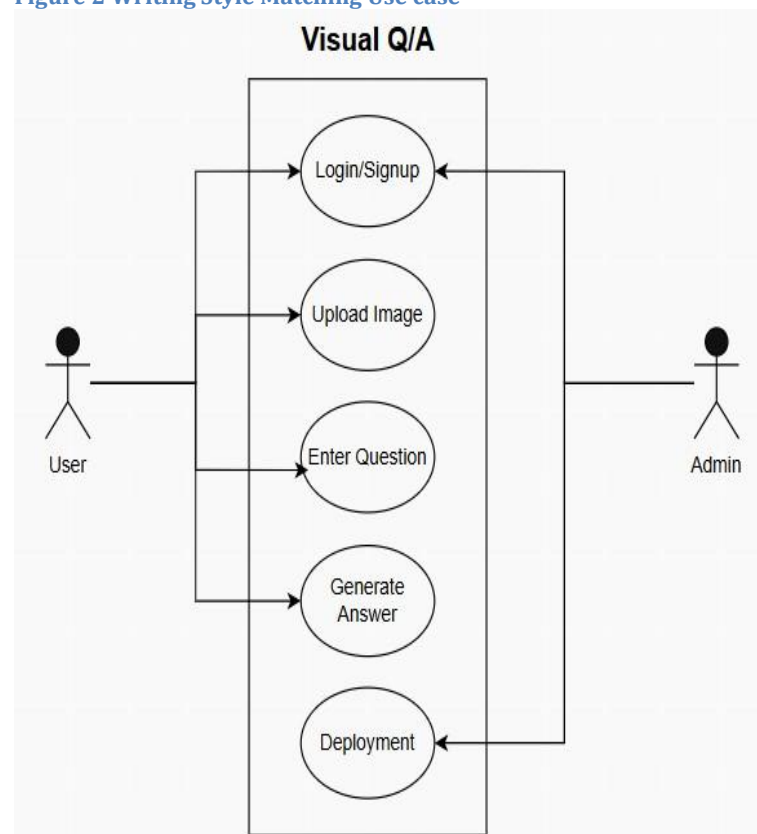


Figure 3 Visual Q/A Usecase

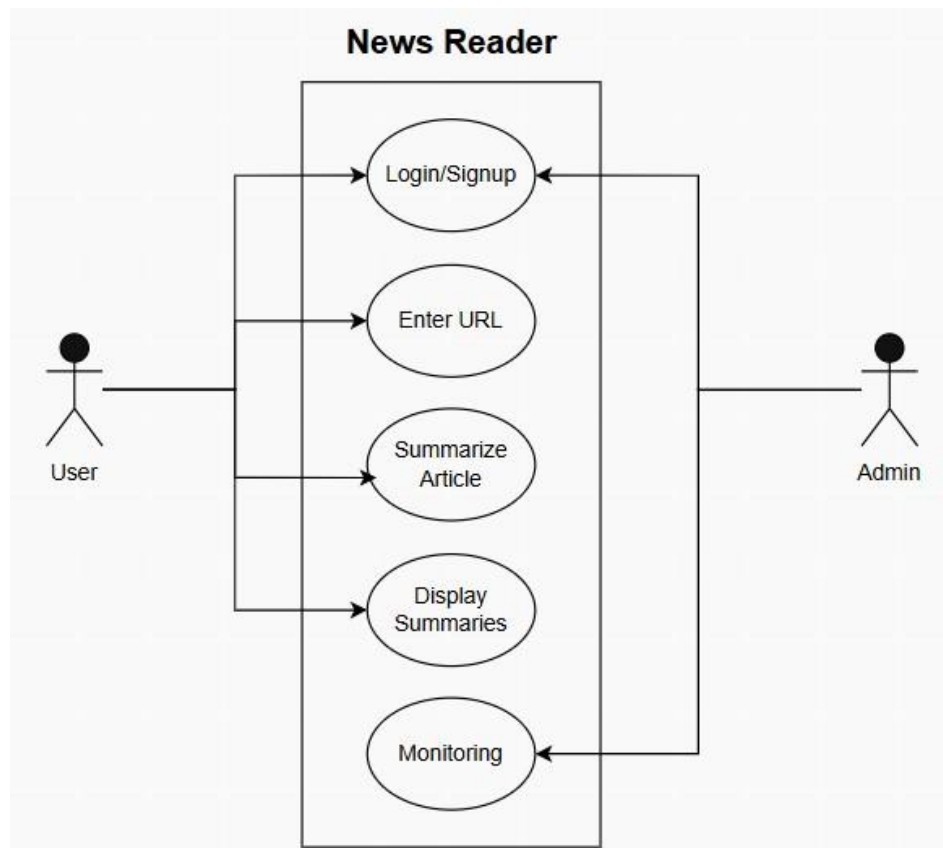


Figure 4 News Reader Usecase

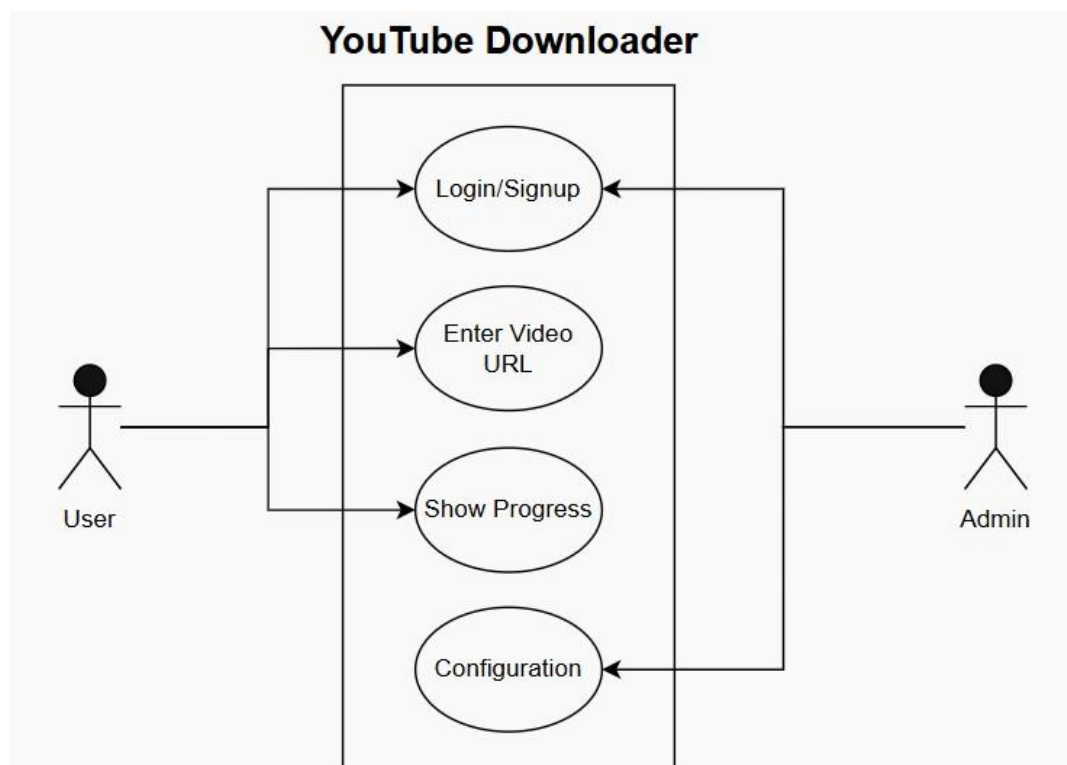


Figure 5 Youtube Downloader use case

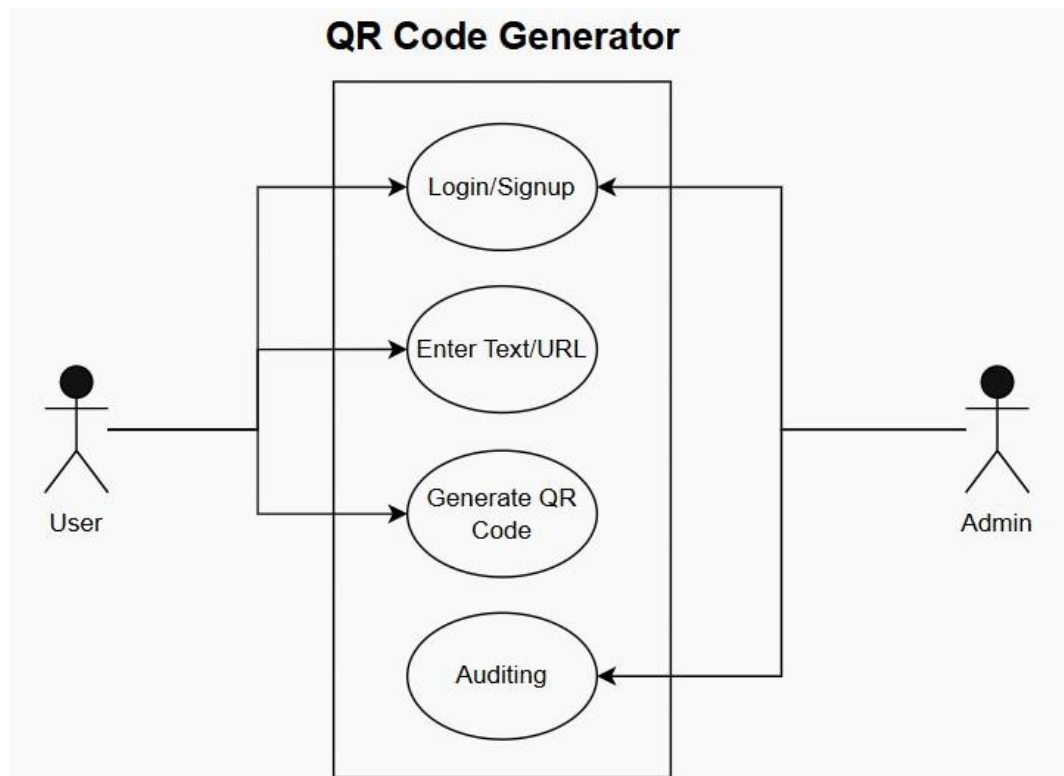


Figure 6 Translator Usecase

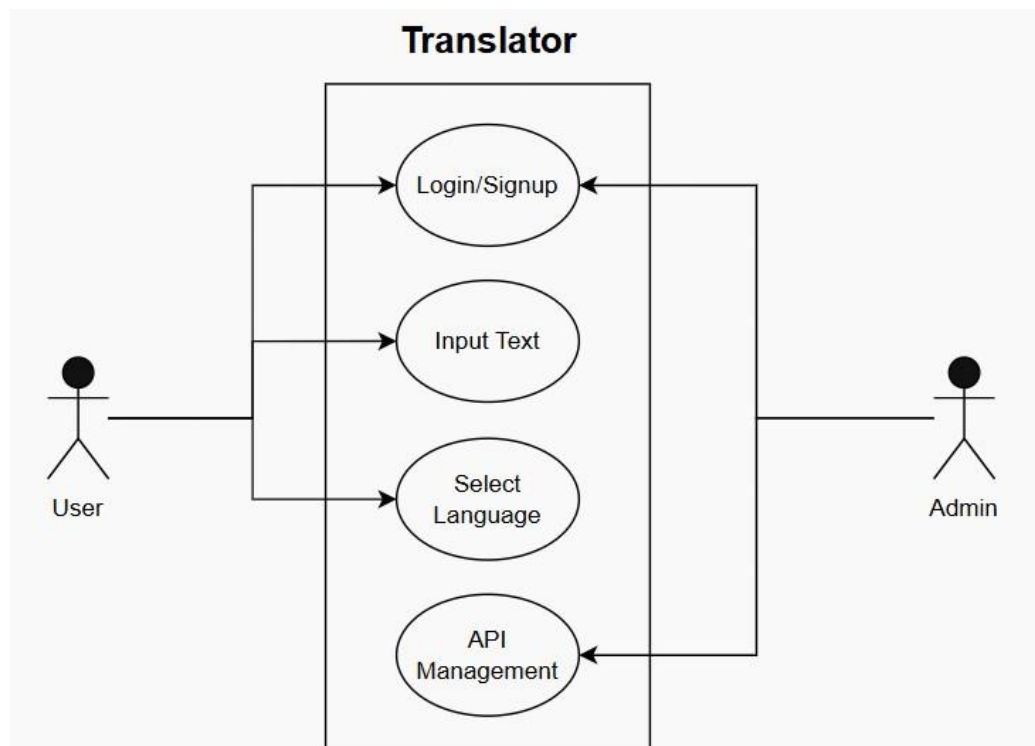


Figure 7 File Converter Usecase

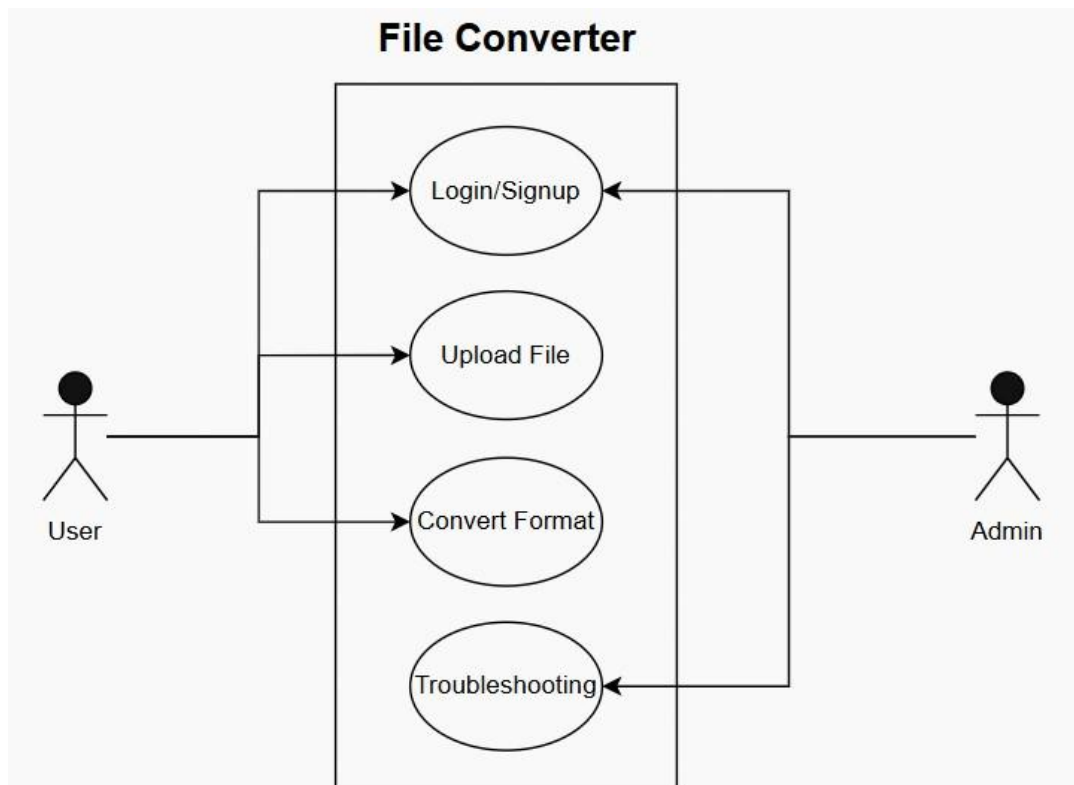


Figure 8 QR Code Generator Use case

3.10. Detailed Use Case

This section describes the major system interactions in the form of use cases. Since the proposed project is a **Multi-Module AI System**, separate use cases are defined for each module, along with a unified high-level use case diagram that illustrates the complete system interaction between the user and all functionalities.

The primary actor in this system is the **End User**, which may include teachers, students, administrative staff, or general application users.

3.10.1. High-Level Use Case Overview

The entire system allows the user to:

1. Upload handwritten PDFs
2. Detect plagiarism between two documents
3. Match writing style
4. Verify signatures
5. Ask questions about images
6. Summarize news
7. Download YouTube videos
8. Generate QR codes
9. Translate text
10. Convert documents

The system responds by performing the requested operation using its respective AI module and providing results back to the user.

3.11. Use Cases by Module

3.11.1 Handwriting Plagiarism Detection – Use Cases

Actor: User

Goal: Compare two handwritten documents for similarity.

Table 5

Use Case	Description
UC-1.1: Upload PDF Documents	User uploads two handwritten PDF files.
UC-1.2: Extract Text	System performs OCR on both PDFs.
UC-1.3: Generate Embedding	System converts extracted text into embedding.
UC-1.4: Compute Similarity	System compares embedding using FAISS/cosine similarity.
UC-1.5: Display Results	System displays plagiarism percentage and matching segments.

3.11.2 Writing-Style / Author Matching – Use Cases

Table 6

Use Case	Description
UC-2.1: Upload Documents	User uploads handwritten PDFs or text.
UC-2.2: Extract Style Features	System analyzes writing patterns, vocabulary, and style.
UC-2.3: Compare Style Profiles	System compares two style signatures.
UC-2.4: Show Style Similarity Score	Results are presented to the user.

3.11.3 Signature Forgery Detection – Use Cases

Table 7

Use Case	Description
UC-3.1: Upload Signature Image	User uploads signature image.
UC-3.2: Preprocess Image	System cleans and resizes the signature.
UC-3.3: Classify Authenticity	Model predicts genuine or forged.

UC-3.4: Display Confidence Score	System shows results with percentage.
----------------------------------	---------------------------------------

3.11.4 Visual Question Answering – Use Cases

Table 8

Use Case	Description
UC-4.1: Upload Image	User uploads an image.
UC-4.2: Enter Question	User types the question about the image.
UC-4.3: Analyze Image & Question	System extracts visual features & interprets question.
UC-4.4: Generate Answer	AI model produces a relevant answer.
UC-4.5: Display Output	Answer shown to the user.

3.11.5 News Reader / Summarizer – Use Cases

Table 9

Use Case	Description
UC-5.1: Fetch Latest News	System retrieves news via API/scraping.
UC-5.2: Summarize Articles	AI summarizes long news into short points.
UC-5.3: Display Summaries	Summaries are shown to user grouped by category.

3.11.6 YouTube Downloader – Use Cases

Table 10

Use Case	Description
UC-6.1: Enter Video URL	User enters YouTube URL.
UC-6.2: Start Download	System uses yt-dlp to download video.
UC-6.3: Show Progress	Real-time progress is displayed.
UC-6.4: Fix Corrupted Files	System attempts repair if file breaks.
UC-6.5: Save History	Previously downloaded videos stored locally.

3.11.7 QR Code Generator – Use Cases

Table 11

Use Case	Description
UC-7.1: Enter Text/URL	User provides data.
UC-7.2: Generate QR Code	System creates QR image.
UC-7.3: Download QR Code	User downloads PNG/SVG.

3.11.8 Translator – Use Cases

Table 12

Use Case	Description
UC-8.1: Input Text	User enters text for translation.
UC-8.2: Select Language	User selects target language.
UC-8.3: Translate	AI model performs translation.
UC-8.4: Display Result	Translated text shown to user.

3.11.9 File Converter – Use Cases

Table 13

Use Case	Description
UC-9.1: Upload File	User uploads PDF or image.
UC-9.2: Convert Format	System converts between PDF ↔ Image or OCR to text.
UC-9.3: Download Output	Converted file available for download.

3.12 Unified Use Case Diagram (Description)

The unified use case diagram includes the following:

Actor:

User

System:

Multi-Module AI Document Processing System

Use Cases:

- Detect Handwriting Plagiarism
- Match Writing Style
- Verify Signatures
- Perform Visual Q/A
- Summarize News
- Download YouTube Videos
- Generate QR Codes
- Translate Text
- Convert Files

The user interacts with each module through the centralized interface. Each module processes user inputs independently and returns results back to the UI.

4. Functional Requirements

Functional requirements define the core behavior, features, and actions the system must perform. Since this project is a **Multi-Module AI Document Processing & Assistance Platform**, each module has its own set of functional requirements, followed by system-wide requirements.

4.1. Handwriting Plagiarism Detection (Core Module)

Upload Handwritten PDFs

The system shall allow users to upload two handwritten PDF documents for comparison.

Extract Text from PDFs

The system shall perform OCR on both documents to extract handwritten text.

Pre-process Extracted Text

The system shall clean text by removing noise, symbols, and OCR errors.

Generate Embedding

The system shall convert text into embedding using Sentence-BERT or similar models.

Compute Similarity Score

The system shall compute similarity using cosine similarity or FAISS.

Display Plagiarism Results

The system shall show plagiarism percentage, similarity highlights, and summary.

4.2. Writing-Style / Author Matching Module

Upload PDFs or Text

The system shall accept handwritten PDFs or extracted text for analysis.

Extract Writing Style Features

The system shall generate linguistic, structural, and embedding-based style signatures.

Compare Style Profiles

The system shall compare two style profiles to evaluate author consistency.

Display Style Similarity Score

The system shall show how similar the writing styles are between documents.

4.3 Signature Forgery Detection Module

Upload Signature Image

User uploads a scanned or photographed signature.

Preprocess Signature

The system shall resize, threshold, and clean the signature image.

Classify Authenticity

The system shall predict whether the signature is genuine or forged.

Show Confidence Result

The system shall display authenticity percentage and reasoning (if available).

4.4. Visual Question Answering (VQA)

Upload Image

The system shall allow uploading an image for analysis.

Enter Question

The system shall accept a question related to the uploaded image.

Extract Visual Features

The AI shall detect objects, text, scenes, and context from the image.

Interpret the Question

The system shall analyze the question using NLP.

Generate Answer

The system shall generate a text-based answer using the VQA model.

4.5. News Reader & Summarizer Module

Fetch Latest News

The system shall retrieve articles from selected news sources.

Summarize Articles

The AI shall summarize long news into short bullet points.

Categorize Articles

The system shall classify summaries by category (sports, tech, politics, etc).

Display Summaries

The user shall view organized summaries on the interface.

4.6. YouTube Downloader Module

Accept YouTube URL

The system shall allow users to input a video link.

Start Download

The backend shall use yt-dlp to download the requested video.

Show Real-Time Progress

The system shall display download percentage and size to the frontend.

Repair Corrupted Files

The system shall attempt automatic repair if the video file is broken.

Store Download History

System shall maintain a list of previously downloaded videos.

4.7. QR Code Generator Module

Input Data or Link

User provides text or URL.

Generate QR Code

System shall create a QR code image instantly.

Download QR Code

User shall download the QR code in PNG or SVG format.

4.8. Translator Module

Input Text

User provides sentences or paragraphs for translation.

Select Language

User chooses target language (e.g., Urdu ↔ English).

Translate Text

System shall translate text with AI/NLP models.

Display Translated Output

System shows neatly formatted translated content.

4.9. File Converter Module

Upload File

System shall accept PDF, images, or text files.

Convert File Format

System shall convert:

- PDF ↔ Image
- Image → Text (OCR)
- Other supported formats

Provide Downloadable Output

The converted file shall be available to download.

4.10. System-Wide Functional Requirements

Unified Dashboard

The system shall provide a dashboard listing all modules.

Error Handling

The system shall detect missing input, invalid formats, and OCR failures.

File Validation

The system shall validate file size, extension, and quality.

Module Communication

Node.js backend shall communicate with Python AI modules using APIs or subprocesses.

Logging

The system shall log key operations (uploads, errors, processing times).

Result Display

Each module must show clean, readable, and accurate results to the user.

5. Non-Functional Requirements

Non-functional requirements define **how the system performs its functions**, ensuring quality, reliability, security, and user satisfaction. These requirements apply to **all modules** of the Multi-Module AI Document Processing & Assistance System, including plagiarism detection, writing-style matching, signature verification, visual Q/A, translation, QR generation, file conversion, news summarization, and YouTube video downloading.

5.1. Performance Requirements

NFR-5.1.1 Processing Time

- The system should process handwritten PDFs (OCR + embeddings + similarity) within **10–25 seconds**.
- VQA image analysis should generate answers within **5–10 seconds**.
- QR code generation and translation should respond **instantly (<2 seconds)**.

NFR-5.1.2 API Response Time

- Communication between Node.js backend and Python AI modules should occur within **1–3 seconds**.

NFR-5.1.3 Efficient Similarity Search

- The system should use FAISS or optimized cosine similarity to ensure similarity search in **under 1 second** for large vector sets.

NFR-5.1.4 Multi-Task Handling

- The system must be capable of running multiple modules independently without affecting performance.

5.2. Reliability Requirements

NFR-5.2.1 Accuracy of AI Models

- Plagiarism detection accuracy should be consistently high for similar writing patterns.
- Signature verification should show stable predictions with confidence scores.
- VQA answers should be logically relevant and contextual.

NFR-5.2.2 System Uptime

- The system should maintain **99% uptime** during operational hours.

NFR-5.2.3 Fault Tolerance

- If a Python process crashes, the system should automatically restart or return a safe fallback response.

NFR-5.2.4 Safe File Handling

- User files must never corrupt, get lost, or overwrite existing files accidentally.

5.3. Usability Requirements

NFR-5.3.1 User-Friendly Interface

- The interface should be modern, intuitive, and require no technical knowledge.
- All modules should follow a consistent UI layout.

NFR-5.3.2 Accessibility

- Results should be easy to read with proper formatting, spacing, and color coding.

NFR-5.3.3 Guidance and Feedback

- Each module should display tooltips, instructions, or placeholder text to guide users.
- Clear error messages should be shown for incorrect inputs.

NFR-5.3.4 Minimal Input Requirements

- The system should require minimum user effort—upload a file or enter text, and the rest should be automated.

5.4. Security Requirements

NFR-5.4.1 Secure File Upload

- Uploaded files must be sanitized before processing to prevent malicious injection.

NFR-5.4.2 Data Privacy

- Handwritten documents, signatures, videos, and text data must not be stored permanently unless required.

NFR-5.4.3 Protected APIs

- APIs should validate all requests and reject unauthorized or malformed inputs.

NFR-5.4.4 Safe Python Execution

- AI processes must run inside isolated or sandboxed environments to avoid OS-level risks.

NFR-5.4.5 No Data Leakage

- Sensitive input such as signatures should be processed in memory and deleted after use.

5.5. Scalability Requirements

NFR-5.5.1 Modular Architecture

- New AI tools should be easily pluggable into the system without redesigning the entire platform.

NFR-5.5.2 Dataset Scaling

- The platform should support large handwritten PDF datasets when deployed in production environments.

NFR-5.5.3 Server Scalability

- If deployed on cloud or VPS, the system should support load balancing and horizontal scaling.

5.6. Maintainability Requirements

NFR-5.6.1 Code Modularity

- Each module must be developed as an independent component to simplify updates.

NFR-5.6.2 Logging & Monitoring

- Important events (errors, downloads, OCR failures, processing times) must be logged for debugging.

NFR-5.6.3 Documentation

- Developers must have clear documentation for APIs, models, file structures, and workflows.

NFR-5.6.4 Extensibility

- AI models should be upgradable (e.g., switching to a newer transformer model) without major system refactoring.

5.7. Portability Requirements

NFR-5.7.1 Cross-Platform Compatibility

- The system should run on Windows, Linux, or cloud environments.

NFR-5.7.2 Browser Compatibility

- The frontend must work smoothly on Chrome, Edge, and Firefox.

NFR-5.7.3 Cloud Deploy ability

- The backend should be deployable on VPS, Docker, or Cloud Run.

5.8. Ethical Requirements

NFR-5.8.1 Fair Use

- AI predictions (plagiarism, signature verification) must not produce harmful or biased outcomes.

NFR-5.8.2 Transparency

- The system should clearly show how similarity or authenticity was determined

6. Design and Architecture

This chapter presents the software design and architectural details of the Multi-Module AI Document Processing & Assistance System. The purpose of this chapter is to describe how the system is structured internally, how components interact, and how different AI modules work together inside a unified platform. This chapter follows the Software Design Description (SDD) guidelines and includes architectural views, system decomposition, component descriptions, data flow, and database design.

6.1. System Architecture Overview

The system is based on a **modular, service-oriented architecture** consisting of:

6.1.1. Node.js Backend (Main Controller Layer)

- Handles API endpoints
- Receives requests from frontend
- Manages file uploads
- Forwards tasks to Python AI modules
- Returns processed results[12]

6.1.2. Python AI Engine (Processing Layer)

Each AI module is implemented in Python and exposed to the Node.js layer. It performs:

- OCR (PDF → Text)
- Embedding generation
- Similarity computation
- Signature classification
- Visual Q/A inference
- Translation using NLP models
- QR generation
- File conversion
- News summarization
- yt-dlp video downloading

6.1.3. Frontend (Presentation Layer)

Built with React / Next.js:

- Provides user interface
- Dashboards for modules
- Result visualization
- Real-time progress bars
- File previews

6.1.4. Local Storage / Dataset Layer

- Temporary storage for uploaded PDFs
- Cache for embedding or downloaded videos
- Log files for debugging and analytics

6.1.5 Architecture Style

- Modular
- Layered
- Extensible (new modules can be plugged in easily)
- API-driven

6.2. High-Level Architecture Diagram (Description)

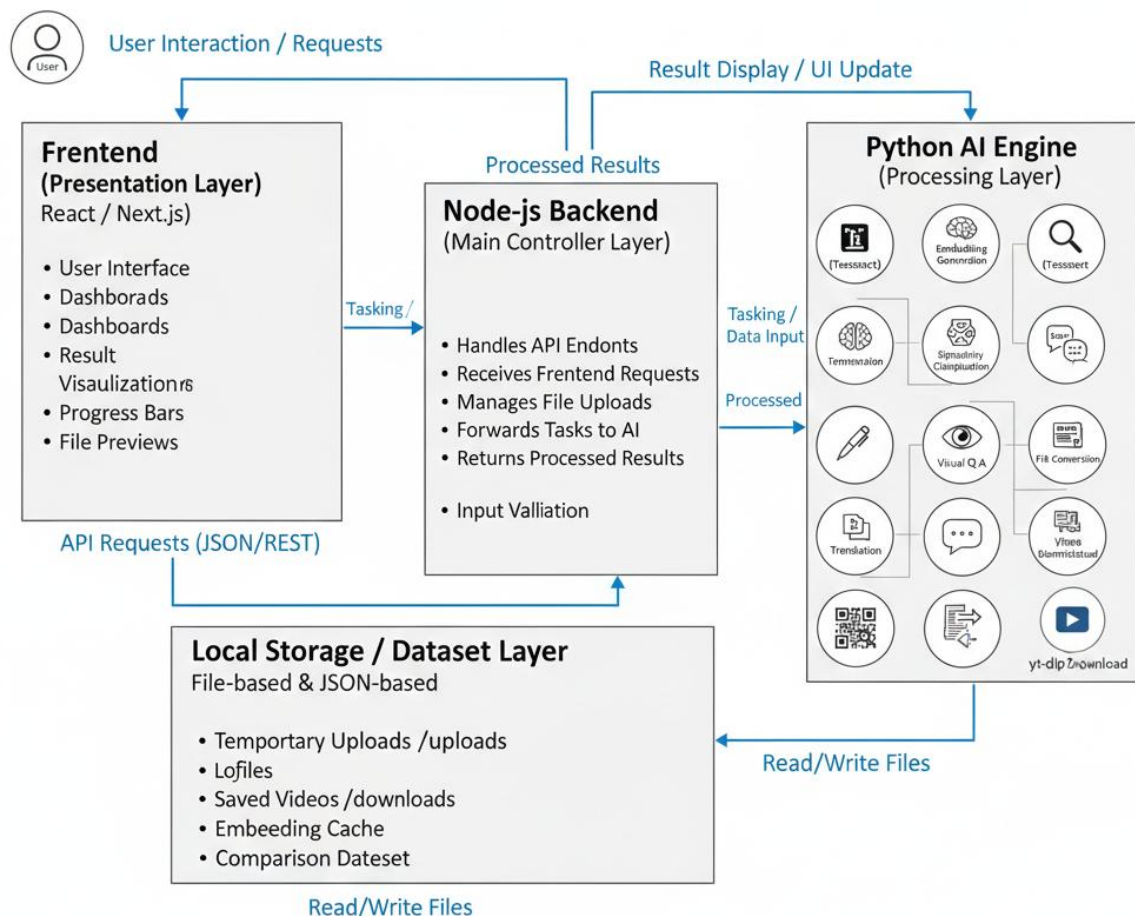


Figure 9 Architecture Diagram

6.2.1 Main Flow:

1. User interacts with frontend
2. Frontend sends request → Node.js backend
3. Node.js:
 - Validates input
 - Stores file temporarily
 - Sends task to Python module
4. Python processes data
5. Python returns output → Node.js → Frontend
6. Frontend displays results

6.3. System Decomposition

The system is decomposed into **9 major modules**, each having its own sub-components.

6.3.1 Handwriting Plagiarism Detection Components

- PDF Upload Handler
- OCR Text Extractor
- Text Preprocessor
- Embedding Generator
- Similarity Engine (FAISS/Cosine)
- Report Formatter

6.3.2 Writing-Style & Author Matching Components

- Text Feature Extractor
- Style Embedding Model
- Author Profile Comparator
- Style Similarity Scoring Unit

6.3.3 Signature Forgery Detection Components

- Signature Preprocessing Unit
- Deep CNN Classification Model
- Authenticity Scoring Engine

6.3.4 Visual Q/A Components

- Image Feature Extractor (Vision Transformer / CNN)
- Question Encoder
- Fusion Model (Image + Text)
- Answer Generator

6.3.5 News Reader Components

- News Fetcher API
- Summarization Model
- Topic Classifier
- UI Renderer for summaries

6.3.6 YouTube Downloader Components

- URL Parser
- yt-dlp Execution Engine
- Progress Logger
- Video Repair System
- Download History Manager

6.3.7 QR Code Generator Components

- Text/URL Handler
- QR Generator Library
- Download Service

6.3.8 Translator Components

- Language Detection Module
- Transformer-based Translation Model
- Output Formatter

6.3.9 File Converter –Components

- Format Detector (Image/PDF/Text)
- PDF-to-Image Converter
- Image-to-Text OCR
- Result Exporter

6.4. Component Interaction Diagram (Description)

6.4.1 Frontend → Backend → Python Interaction

1. User Action

- Upload file / Enter text / Provide URL / Ask question

2. Node.js

- Validates, stores & forwards request

3. Python Script

FYP Project Report

- Runs AI inference
- Sends output back

4. Node.js

- Packages results

5. Frontend

- Displays formatted output

This flow repeats for every module independently.

6.5. Data Flow Diagrams (DFD)

Level 0 DFD

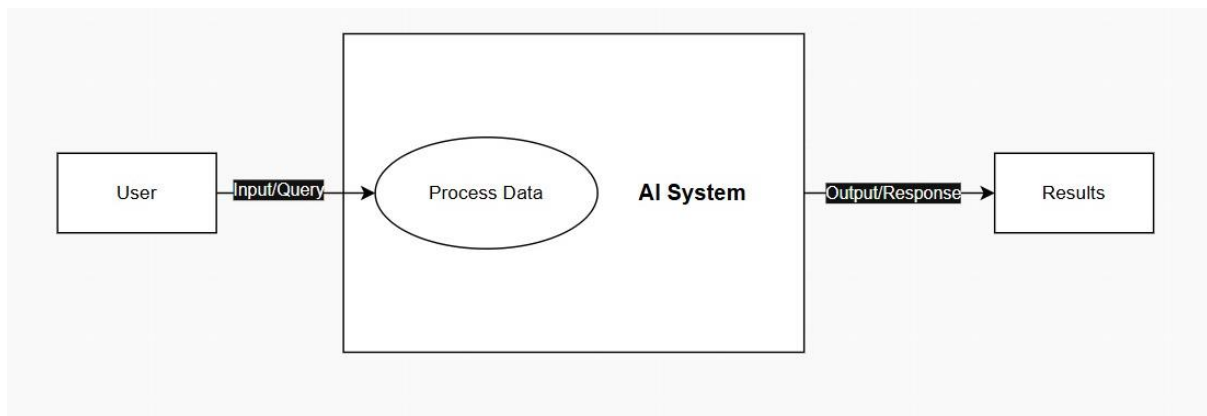


Figure 10 Level 0 DFD

Level 1 DFD (Per Module Example)

For plagiarism detection:

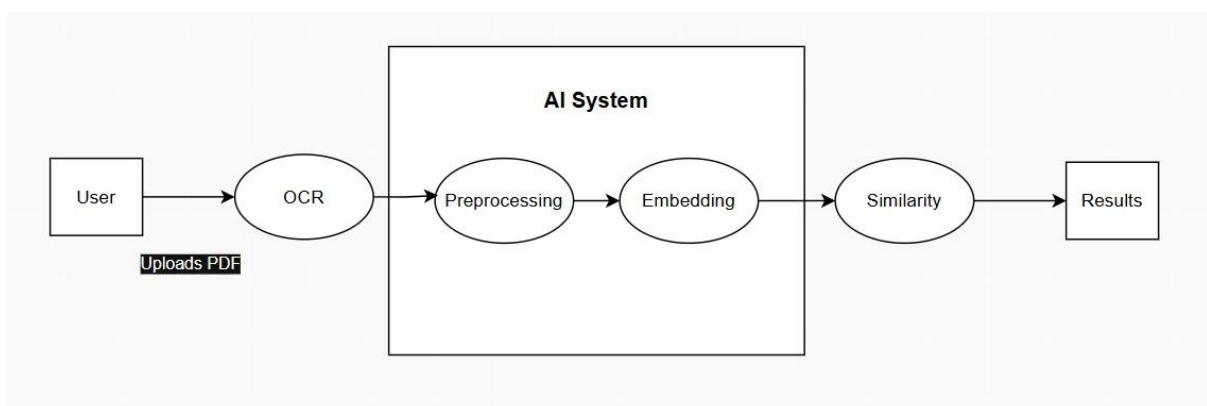


Figure 11 Level 1 DFD Plagiarism Detection

For Visual Q/A:

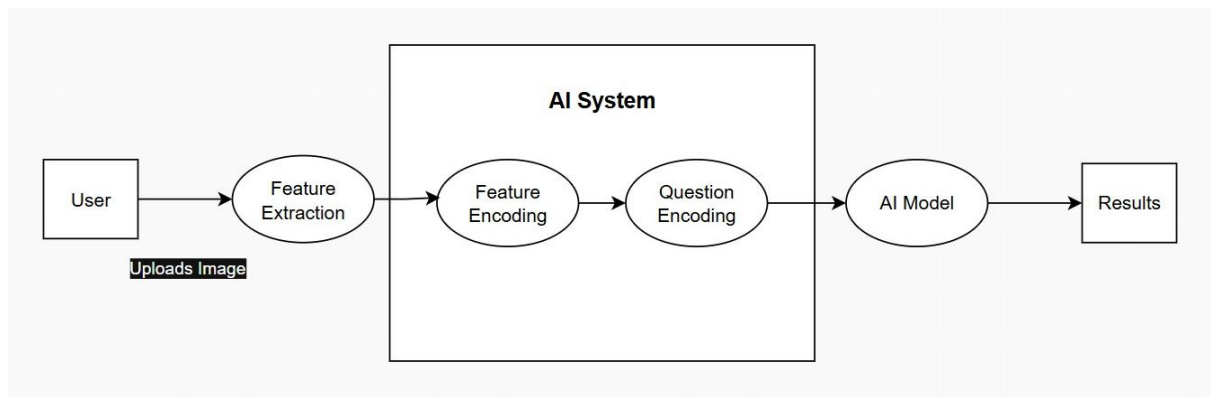


Figure 12 Level 1 DFD Visual Q/A

For Writing Style Matching:

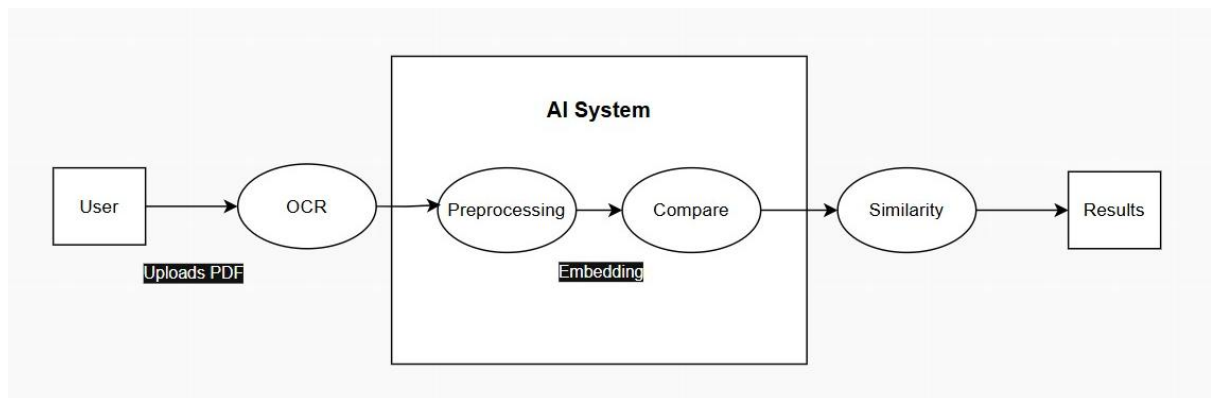


Figure 13 Level 1 DFD Writing Style Matching

For News Reader:

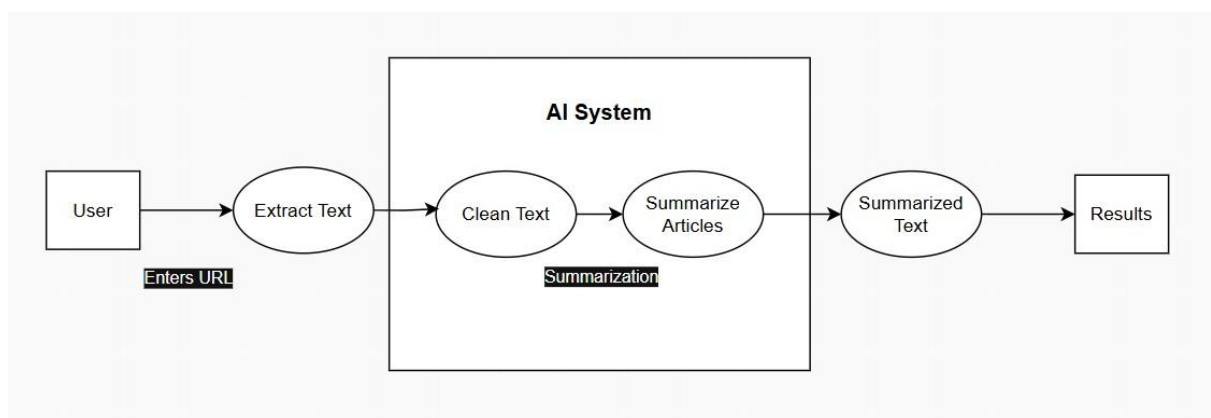


Figure 14 Level 1 DFD News Reader

FYP Project Report

For Youtube Downloader:

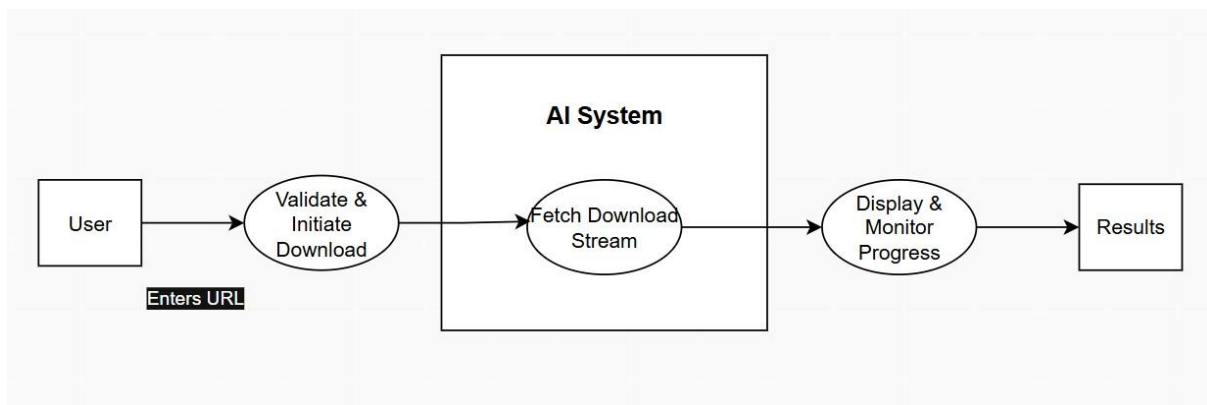


Figure 15 Level 1 DFD Youtube Downloader

For Translator:

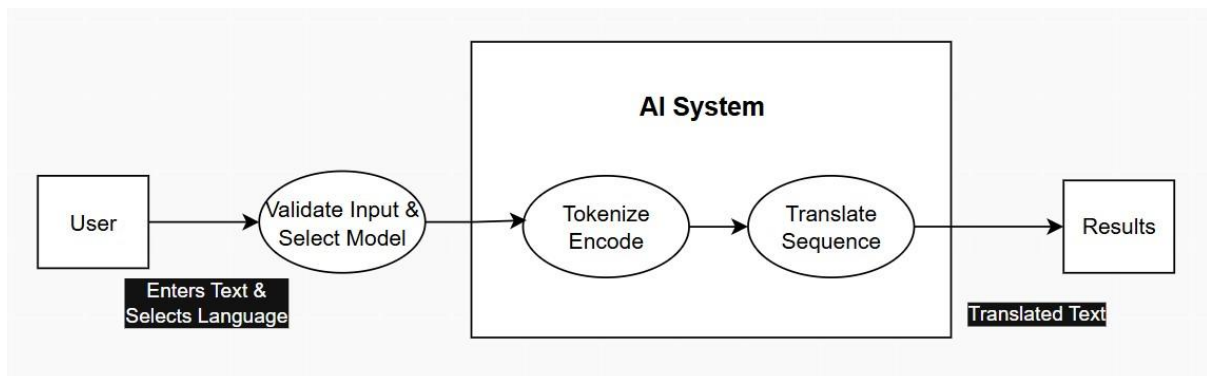


Figure 16 Level 1 DFD Translator

For File Converter:

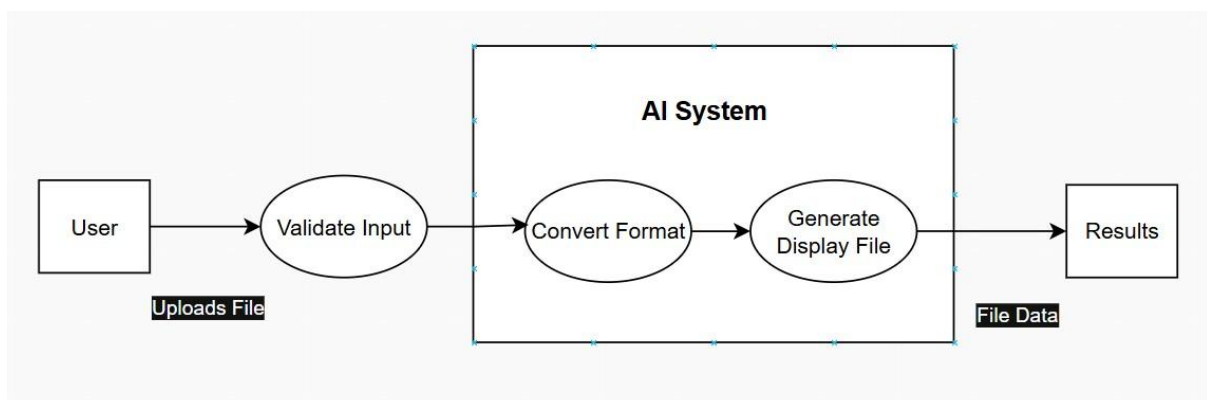


Figure 17 Level 1 DFD File Converter

6.6. Database / File Structure Design

This system uses **file-based and JSON-based storage**, not SQL, due to the nature of uploaded files.

Directories

- /uploads/ → temporary files
- /downloads/ → saved videos
- /qr/ → QR code PNGs
- /embeddings/ → stored vectors (optional)
- /logs/ → errors & process logs

6.7. User Interface Design Overview

6.7.1. Main UI Modules

- Dashboard (all tools visible)
- Plagiarism detection page
- VQA page
- Signature verification page
- QR generator page
- Translator page
- News reader page
- YouTube downloader page
- File converter page

6.7.2. UI Design Principles

- Minimalist
- Purple gradient theme
- Icons per module
- Responsive layout
- Real-time updates (progress bars)

6.8. Error Handling & Exception Design

- PDF upload failures → Show “Invalid file”
- Python crash → Auto retry or safe response
- OCR errors → Display warning message
- Video download failed → Provide alternate resolutions
- Signature too blurry → Ask for better image

6.9. Security Design

- Input sanitization
- No permanent storage of sensitive files

- API rate limiting
- Enforced file type restrictions (PDF, PNG, JPG)
- Sandboxed Python execution

6.10. Scalability & Extensibility Design

- New modules can be added under /ai_modules/ folder
- Node.js dynamically routes module requests
- Python functions independent & plug-in friendly
- No modification needed in other modules when adding a new one

6.11. Technology Stack Overview

6.11.1. Frontend

- React / Next.js
- Tailwind
- Axios

6.11.2. Backend

- Node.js
- Express.js
- Multer

6.11.3. AI Layer (Python)

- PyTorch
- HuggingFace Transformers
- Sentence-Transformers
- FAISS
- OpenCV
- Tesseract / EasyOCR
- yt-dlp
- qrcode
- langdetect / translation models

7. System Architecture

The proposed system follows a **layered and modular architecture** designed to support multiple AI-based tools within a single unified platform. The architecture separates the presentation layer, application logic, and AI processing layer so that each module (Plagiarism Detection, Writing-Style Matching, Signature Forgery Detection, Visual Q/A, News Reader, YouTube Downloader, QR Code Generator, Translator, File Converter) can be developed, tested, and extended independently.

At a high level, the system consists of four main layers:

1. **Client / Presentation Layer (Frontend)**
2. **Application Layer (Node.js Backend)**
3. **AI Processing Layer (Python Services)**
4. **Storage and Utility Layer**

7.1. Client / Presentation Layer

The presentation layer is implemented using **React / Next.js** and provides the graphical user interface through which users interact with the system. The main responsibilities of this layer are:

- Displaying the **dashboard** containing tiles for all modules (Plagiarism Detection, Visual Q/A, QR Code Generator, Translator, etc.).
- Handling **user inputs** such as file uploads, text entries, questions, URLs, and translation requests.
- Sending requests to the Node.js backend using **HTTP / REST APIs**.
- Rendering **results** returned from the backend, such as plagiarism scores, similarity reports, translated text, generated QR codes, news summaries, or download progress.
- Showing **validation messages**, loading states, and error notifications to improve usability.

The frontend is completely independent of the underlying AI logic; it only knows which API endpoint to call for each module.

7.2. Application Layer (Node.js Backend)

The Node.js backend acts as the **central controller** of the system. It exposes RESTful APIs for each module and coordinates communication between the frontend and the Python AI services.

Key responsibilities:

Routing and Module Dispatching

- Each module has specific endpoints, e.g.
 - /api/plagiarism/check
 - /api/vqa/ask
 - /api/signature/verify
 - /api/qr/generate
 - /api/translate
 - /api/youtube/download
- Based on the request, Node.js forwards data to the corresponding Python script or service.

File Handling

- Uses Multer to handle upload of PDFs, images, and other files.
- Temporarily stores files in module-specific directories (e.g. /uploads/plagiarism, /uploads/signature).

Process Management / Inter-Service Communication

- Invokes Python scripts via child processes or HTTP (if Python is exposed as a micro service).
- Sends input parameters (file paths, text, URLs) and receives AI results in JSON form.

Error Handling & Validation

- Validates user inputs (file size, type, URL format).
- Catches errors from Python processes and responds with meaningful error messages to the frontend.

This layer decouples the UI from AI models and makes it easy to plug in new modules in the future.

7.3. AI Processing Layer (Python Services)

The AI Processing Layer contains **all core intelligence of the system** and is implemented in Python. Each module is implemented as a separate script or service, but they follow a similar pattern: receive input → process using AI/ML → return structured output.

Main components per module:

Plagiarism & Writing-Style Matching Service

- OCR engine (e.g. Tesseract / EasyOCR) for converting handwritten PDFs to text.
- Preprocessing pipeline for cleaning extracted text.
- Sentence-Transformers / embeddings model for vector representation.
- FAISS or cosine similarity engine to compute plagiarism and style similarity scores.
- Logic to build detailed similarity reports.

Signature Forgery Detection Service

- Image preprocessing (grayscale, resize, and thresholding).
- CNN or deep learning model trained on genuine and forged signatures.
- Output: class label (genuine / forged) + confidence score.

Visual Q/A Service

- Image feature extractor (e.g. vision transformer / CNN).

FYP Project Report

- Question encoder (text model).
- Fusion model to combine visual and textual information and generate an answer.

News Reader & Summarizer Service

- News API / scraping client.
- Summarization model to condense long articles.
- Optional topic classifier to tag categories.

YouTube Downloader Service

- yt-dlp wrapper for downloading videos.
- Logic to stream progress updates back to Node.js.
- Optional post-processing/repair of downloaded files.

QR Code Generator Service

- Uses a QR generation library to create QR code images from text or URLs.

Translator Service

- Transformer-based translation model (e.g. Marian / mBART / other).
- Handles language detection and translation.

File Converter Service

- PDF ↔ Image conversion
- Image → Text via OCR
- Returns converted files or text.

Each Python module is **loosely coupled** and can be started, stopped, or upgraded independently without affecting other modules.

7.4. Storage and Utility Layer

This layer provides temporary and persistent storage for files, logs, and (optionally) embedding.

File Storage

- Uploaded PDFs, images, and temporary files stored in structured folders.

Downloads Folder

- Stores downloaded YouTube videos and generated QR codes.

Optional Embedding Storage

- Saved vector representations for faster re-comparison (if needed).

Logs

- Stores error logs, processing times, and debugging information.

A full database is not mandatory for the core prototype, but the architecture allows integration of MongoDB or another DBMS in future versions.

7.5. Architectural Characteristics

- **Modularity:** Each AI feature is treated as a separate module, which simplifies maintenance, testing, and extension.
- **Scalability:** New modules (e.g., future AI tools) can be added by creating a new Python script and a corresponding Node.js route and UI card.
- **Separation of Concerns:**
 - Frontend → user interaction
 - Node.js → orchestration and API management
 - Python → AI processing
- **Extensibility:** Models can be upgraded (e.g., using a better transformer model) without changing the entire system.
- **Reusability:** Shared utilities such as OCR, embedding generation, and file handling can be reused across multiple modules.

7.6. Data Representation [Diagram + Description]

The Data Representation Diagram & its description are given below:

7.6.1. Data Representation

Data representation defines **how information flows**, how it is **structured**, and how different modules in the system **exchange and process data**. Since your FYP is a multi-module AI system, proper structuring of input data, intermediate data, and output data is essential for consistent behavior and modularity.

7.6.1.1. Diagram

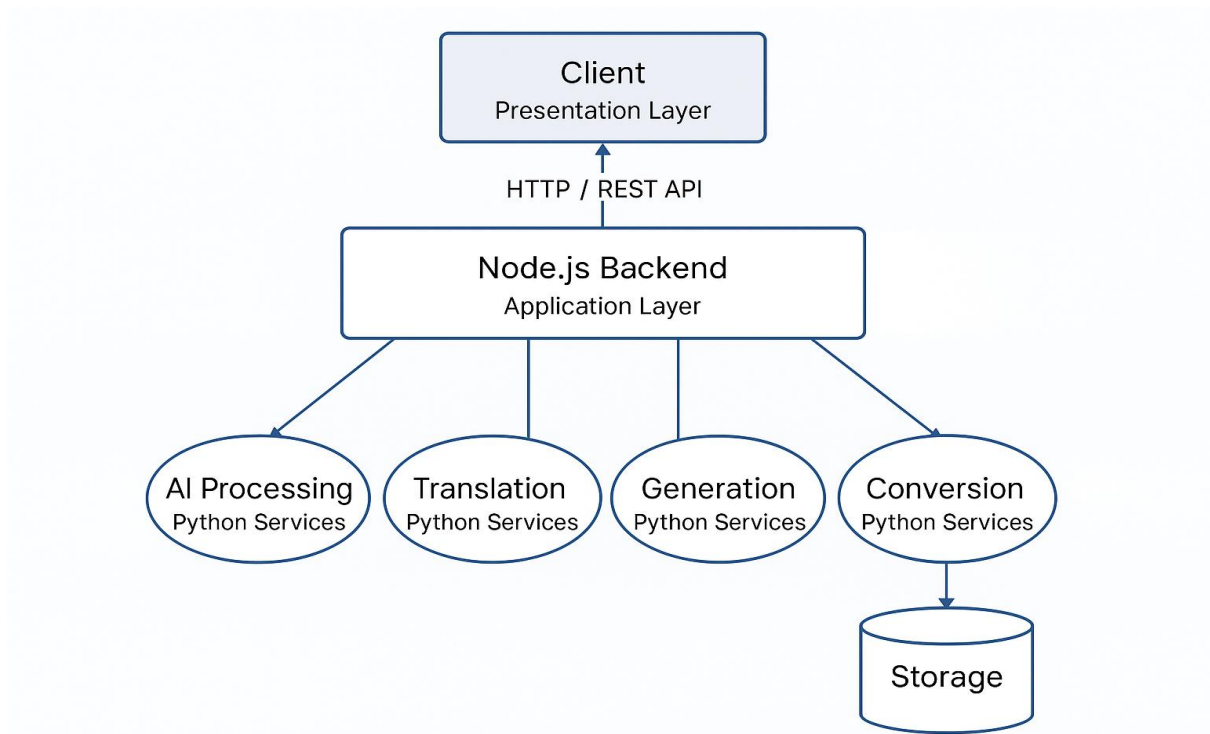


Figure 18 Data Representation Diagram

7.6.2. Diagram: Data Representation Architecture

The diagram illustrates the **flow of data** from the frontend to the backend and then into Python AI services, before being sent back to the user as results.

It contains:

- **Client (Frontend)**
- **Node.js Backend**
- **Python Services (AI Processing, Translation, Generation, Conversion)**
- **Storage Layer**

This visually shows *how your system handles data from start to finish for all modules*.

7.6.3. Description of Data Representation

7.6.3.1. Input Data Representation

Each module receives a specific kind of input:

Handwriting Plagiarism / Writing-Style Module

- PDF files (uploaded by the user)

FYP Project Report

- Internal representation:
 - Converted images (PDF → images)
 - OCR-extracted text
 - Cleaned text arrays
 - Embedding vectors (float32 vectors)

Signature Forgery Detection

- Image file (PNG/JPG)
- Internal representation:
 - Grayscale image matrix
 - Preprocessed tensor for deep learning model
 - Classification output (genuine/forged)

Visual Q/A

- Image (as tensor)
- User question (as tokenized text)
- Internal representation:
 - Vision embedding
 - Text embedding
 - Attention-fused representation

News Summarizer

- Article text (string)
- Internal representation:
 - Token IDs
 - Summary text

YouTube Downloader

- URL (string)
- Internal representation:
 - yt-dlp output stream
 - Progress JSON
 - MP4 file bytes

QR Code Generator

- Text/URL (string)
- Internal representation:

FYP Project Report

- Encoded QR matrix
- PNG byte object

Translator

- Input text string
- Language pair
- Internal representation:
 - Tokenized sequence
 - Translated sequence

File Converter

- Image/PDF file
- Internal representation:
 - Raster image array
 - Converted file format data

7.6.3.2. Processing Data Representation

Node.js Backend

Node.js represents data using:

- JSON for communication
- Streams for file upload/download
- File buffers for temporary storage

Python AI Engine

Python modules convert incoming data into specific formats:

Table 14

Module	Internal Data Format
OCR	Images, text arrays
Embedding	384–768 dimensional vector arrays
Signature Model	Normalized RGB/gray tensors
VQA Model	Vision tensors + text token IDs
Translation	Tokenized sequences
QR Generator	Encoded binary QR matrix

Module	Internal Data Format
File Converter	Rasterized images / PDFs

All these are returned back to Node.js as **JSON objects or binary data**, depending on the module.

7.6.3.3. Output Data Representation

Each module produces different output formats:

Plagiarism Detection

```
{
  "similarityScore": 87,
  "matchedSentences": [...],
  "embeddingStats": {...}
}
```

Writing Style Matching

```
{
  "styleSimilarity": 79,
  "profileComparison": {...}
}
```

Signature Forgery Detection

```
{
  "result": "Forged",
  "confidence": 0.92
}
```

Visual Q/A

```
{
  "answer": "The image shows a boy holding a football."
}
```

News Reader

```
{
  "summaries": ["...", "..."],
  "categories": ["Politics", "Sports"]
}
```

YouTube Downloader

```
{
  "progress": 65,
  "filePath": "/downloads/video.mp4"
}
```

QR Code Generator

- Returns PNG image bytes

Translator

```
{
  "translatedText": "یہ ایک مثال ہے"
}
```

}

File Converter

- PDF or Image binary output

7.6.3.4. Storage Layer Representation

The storage layer contains:

Folders

- /uploads/
- /downloads/
- /qr_codes/
- /converted/
- /logs/

File types

- .pdf, .jpg, .png, .txt, .mp4, .json

No database required (unless added later)

- The system uses structured file-based storage.

7.6.3. Process Flow/Representation

The process flow describes **how data moves** through the system and how different components interact during execution. Since this is a multi-module AI platform, each module follows a similar high-level pattern but has its own specialized internal flow. The process flow ensures clear understanding of each module's internal operations, data transformations, and module interactions.

Below is the general system process flow followed by module-specific flows.

7.6.4. General System Process Flow

Step 1. User Interaction

The user selects a module (Plagiarism, VQA, QR Generator, Translator, etc.) on the frontend dashboard.

Step 2 . Input Submission

User provides:

- A file (PDF/image)
- A text input (URL/question/text)
- A selection (language, module type)

Step 3 . Request Sent to Node.js Backend

Frontend sends input to backend via **REST API**.

Example:

POST /api/plagiarism/check

Step 4. Backend Processing

Node.js:

- Validates input
- Stores file temporarily
- Forwards processed data to Python module via `child_process` or internal API

Step 5. Python AI Module Execution

Python module:

- Processes input using ML/DL algorithms
- Generates output such as embeddings, answers, summaries, translations, QR codes, or MP4 files

Step 6. Return Response

Python sends result → Node.js → Frontend

Response includes:

- JSON data
- File (image, video, PDF)
- Structured results

Step 7. Display to User

The frontend renders:

- Plagiarism %
- VQA answer
- QR code
- Summary
- Download progress
- Converted file

7.6.5. Module-Specific Process Flows

7.6.5.1. Handwriting Plagiarism Detection (Process Flow)

1. User uploads **two handwritten PDFs**
2. Node.js saves PDFs to /uploads/plagiarism
3. Node.js triggers Python script

4. Python:

- Converts PDF → images
- Performs OCR on each page
- Preprocesses extracted text
- Generates Sentence-BERT embeddings
- Computes similarity (FAISS or cosine)
- Creates result object {similarity%, matchedText, stats}

5. Node.js returns results to frontend

6. UI displays similarity score + details

7.6.5.2. Writing-Style Matching (Process Flow)

1. User uploads handwritten PDFs or text

2. Node.js forwards input to Python

3. Python:

- Extracts linguistic features
- Generates writing-style embeddings
- Compares patterns (sentence flow, vocabulary, structure)

4. Python returns style similarity score

5. Frontend displays “Author similarity %”

7.6.5.3 Signature Forgery Detection (Process Flow)

1. User uploads signature image

2. Node.js saves file

3. Python script:

- Converts to grayscale
- Preprocesses (resize, threshold)
- Passes to CNN model
- Predicts **Genuine / Forged**

4. Response returned to UI

7.6.5.4 Visual Question Answering (Process Flow)

1. User uploads image + question

2. Node.js passes both to Python

3. Python:

- Encodes question using NLP
- Extracts image features using CNN/Vision Transformer
- Combines image + text in VQA model
- Produces answer (string)

4. Node.js sends answer to frontend

7.6.5.5 News Reader / Summarizer (Process Flow)

1. User opens News Reader module
2. Node.js calls Python script to fetch news via API
3. Python:
 - Downloads articles
 - Summarizes them using NLP model
 - Categorizes them
4. Frontend shows summarized articles

7.6.5.6 YouTube Downloader (Process Flow)

1. User pastes YouTube URL
2. Node.js calls Python script with URL
3. Python (yt-dlp):
 - Starts download
 - Reports progress (%)
 - Repairs file if corrupted
4. Node.js streams progress back to UI
5. Video saved to downloads/
6. Frontend shows “Download complete” + file link

7.6.5.7 QR Code Generator (Process Flow)

1. User enters text/URL
2. Node.js passes string to Python
3. Python generates QR image
4. Node.js returns PNG base64 or file path
5. UI displays QR code

7.6.5.8 Translator (Process Flow)

1. User provides text + target language
2. Node.js sends data to Python
3. Python:
 - Tokenizes text
 - Uses transformer model to translate
 - Returns translated text
4. UI displays translated output

7.6.5.9 File Converter (Process Flow)

1. User uploads PDF/image
2. Node.js stores file
3. Python:
 - Converts PDF → images OR
 - Image → text via OCR OR
 - Image → PDF
4. Node.js returns converted file to UI
5. User downloads the result

7.6.6. Cross-Module Process Flow Characteristics

- Each module follows the **same top-level architecture** (Frontend → Backend → Python → Backend → Frontend).
- All Python modules are **independent**, allowing easy addition of new ones.
- System is designed using:
 - Loose coupling
 - Separation of concerns
 - Reusable AI pipelines
- File-based storage ensures simplicity and speed.

7.6.7. Diagram (Process Flow Representation)

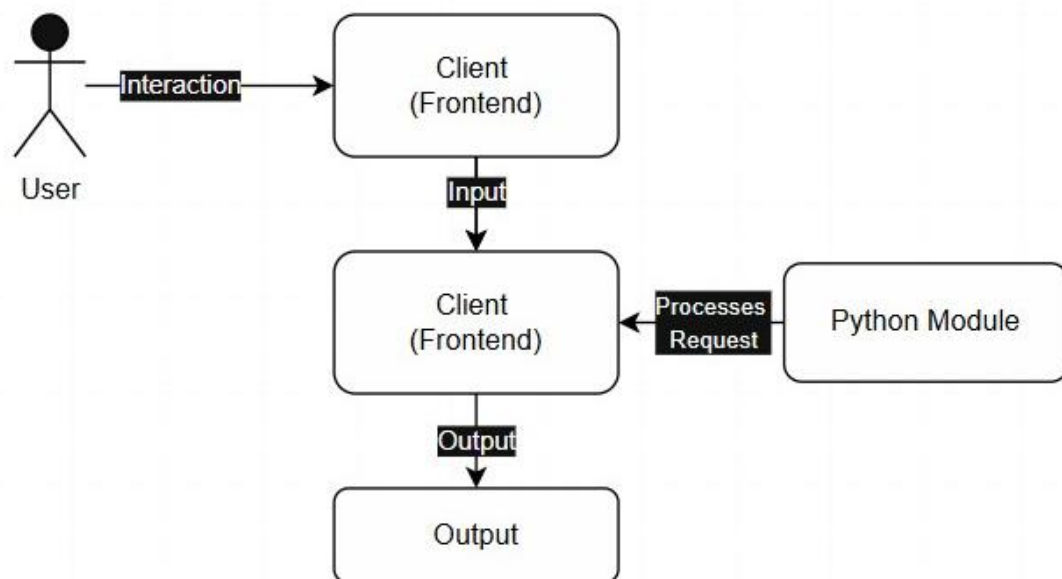


Figure 19 Process Flow Diagram

7.7. Design Models [along with descriptions]

This section describes the different design models used to represent the structure and behavior of the proposed **Multi-Module AI Document Processing & Assistance System**. The models provide different views of the system such as interaction, structure, components, and deployment.

The main models used are:

- Use Case Model
- Class Model
- Sequence Model
- Activity Model
- Component Model
- Deployment Model

7.7.1. Use Case Model

The **Use Case Model** represents the interaction between the **User** and the various modules of the system.

- **Actor:** User (teacher, student, admin, or general user).

Main Use Cases:

- Detect Handwriting Plagiarism
- Match Writing Style
- Verify Signature
- Perform Visual Q/A
- Read & Summarize News
- Download YouTube Video
- Generate QR Code
- Translate Text
- Convert Files

The use case diagrams (one per module) show the **User** actor connected to the corresponding use cases such as Upload PDFs, Compute Similarity, Upload Image, Generate Answer, Enter URL, Show Progress, etc.

These models help to clearly define **what** the system does from the user's point of view.

7.7.2. Class Model

The **Class Model** represents the main logical classes and their relationships. At a high level, the system can be represented using the following core classes:

- **User**

Attributes: userId, name, role

FYP Project Report

Responsibilities: initiates operations, provides inputs.

- **ModuleManager**

Attributes: availableModules

Methods: invokeModule(), getStatus()

Coordinates which module is called based on user selection.

- **PlagiarismService**

Attributes: ocrEngine, embeddingModel

Methods: extractText(), generateEmbeddings(), computeSimilarity()

- **StyleMatchingService**

Methods: extractStyleFeatures(), compareStyles()

- **SignatureService**

Methods: preprocessSignature(), predictAuthenticity()

- **VQAService**

Methods: encodeImage(), encodeQuestion(), generateAnswer()

- **NewsService**

Methods: fetchArticles(), summarizeArticle()

- **YouTubeService**

Methods: downloadVideo(), getProgress(), repairFile()

- **QRCodeService**

Methods: generateQRCode()

- **TranslatorService**

Methods: translateText()

- **FileConverterService**

Methods: convertPDFToImages(), convertImageToText(), exportFile()

- **FileStorage**

Methods: saveFile(), getFile(), deleteFile()

Relations:

- User → interacts with ModuleManager
- ModuleManager → uses individual *Service classes
- Each *Service may depend on FileStorage and external libraries/models.

This model shows the **static structure** of the system and how responsibilities are divided.

7.7.3. Sequence Models

Sequence diagrams describe **how objects interact over time** for specific scenarios.

7.7.3.1 Plagiarism Detection Sequence

Objects: User → Frontend → NodeBackend → PlagiarismService (Python) → Frontend

Flow:

1. User selects “Plagiarism Detection” and uploads two PDFs.
2. Frontend sends POST /api/plagiarism/check to NodeBackend.
3. NodeBackend saves files and calls PlagiarismService.
4. PlagiarismService:
 - extracts text
 - generates embeddings
 - computes similarity
 - returns result JSON
5. NodeBackend sends response to Frontend.
6. Frontend displays similarity percentage and details.

7.7.3.2 YouTube Downloader Sequence

Objects: User → Frontend → NodeBackend → YouTubeService → FileStorage → Frontend

Flow:

1. User pastes YouTube URL.
2. Frontend calls backend API with URL.
3. NodeBackend calls YouTubeService (yt-dlp).
4. YouTubeService downloads video and periodically sends progress.
5. NodeBackend forwards progress to frontend via polling or websockets.
6. After completion, video path is stored in FileStorage and download link shown to user.

Sequence models help describe the **dynamic behavior** for key use cases.

7.7.4. Activity Models

Activity diagrams model the **workflow** of each module.

4.1 Plagiarism Detection Activity

Main activities:

1. Start
2. Upload PDFs
3. Validate files
4. Run OCR on both documents
5. Clean and preprocess text
6. Generate embeddings
7. Compute similarity score
8. Prepare report
9. Display result
10. End

Branches:

- If file validation fails → show error and stop.
- If OCR fails → show warning and stop.

4.2 Visual Q/A Activity

1. Start
2. Upload image
3. Enter question
4. Validate inputs
5. Extract image features
6. Encode question
7. Run VQA model
8. Output answer
9. Display answer
10. End

Activity models emphasize **decision points, loops, and parallel steps** in each workflow.

7.7.5. Component Model

The **Component Diagram** represents physical and logical components and their interfaces.

Main components:

- **Frontend Component**
 - React/Next.js app

- Uses HTTP interfaces to backend.
- **API Gateway / Node.js Backend Component**
 - Express server
 - Exposes REST APIs for each module
 - Depends on Python components.
- **Python AI Components**
 - PlagiarismComponent
 - StyleMatchingComponent
 - SignatureComponent
 - VQAComponent
 - NewsComponent
 - YouTubeComponent
 - QRCodeComponent
 - TranslatorComponent
 - FileConverterComponent
- **Storage Component**
 - Local file system (or future DB).

Interfaces:

- Frontend ↔ Backend: REST/JSON
- Backend ↔ Python: child process / internal HTTP / IPC
- Backend ↔ Storage: file system API

This model helps explain how the system is **physically organized** into deployable pieces.

7.7.6. Deployment Model

The **Deployment Diagram** shows how software components map to hardware nodes.

Typical deployment:

- **Client Node**
 - User's browser accessing frontend.
- **Application Server**
 - Hosts React/Next.js build (static)
 - Runs Node.js backend
 - Manages API routing

FYP Project Report

- **AI Processing Server (can be same or separate)**
 - Runs Python environment
 - Contains all AI modules and models
 - May use GPU or Colab during training
- **Storage Node**
 - Local disk or network-attached storage for uploads, downloads, logs.

Connections:

- Browser ↔ Application Server: HTTPS
- Application Server ↔ AI Server: HTTP / local IPC
- Application Server ↔ Storage: file system or network storage.

The deployment model illustrates how the system can scale from a single machine setup (development) to a multi-server production environment.

8. Implementation

This chapter explains how the Multi-Module AI Document Processing & Assistance System was implemented. It provides a detailed, technical-level overview of all modules, internal workflows, back-end processing, and integration mechanisms. UML diagrams (described where applicable) are used to illustrate interactions and internal module behavior.

No source code is included, in accordance with FYP report standards.

8.1. Implementation Overview

The system follows a **client–server, multi-module architecture**.

Implementation is divided into:

- **Frontend (React / Next.js)**
- **Backend (Node.js / Express.js)**
- **AI Processing Layer (Python scripts and ML models)**
- **Storage Layer (Local directories for uploads, outputs, logs)**

The design ensures modularity: each feature (plagiarism, VQA, signature detection, etc.) operates independently while sharing a unified interface and communication structure.

8.2. Frontend Implementation

The frontend was implemented using **Next.js with React**, styled using Tailwind CSS.

Major UI components include:

8.2.1 Dashboard Screen

- Displays all modules: Plagiarism Detection, Writing-Style Matching, Signature Forgery Detection, VQA, YouTube Downloader, News Reader, QR Generator, Translator, File Converter.
- Each module is represented by a clickable tile with icon + label.

8.2.2 Input Interfaces

Each module has a dedicated page:

- PDF upload fields
- Image upload components
- Text input areas
- URL input
- Dropdown language selectors
- Progress bars (YouTube & file upload)

8.2.3 API Service Layer

Frontend communicates with backend using:

- `fetch()` or `Axios` calls
- Routes such as `/api/plagiarism/check`, `/api/youtube/download`, etc.

8.3. Backend Implementation (Node.js)

Backend is implemented using **Express.js** and acts as the **central controller**.

Responsibilities:

- Validating frontend requests
- Handling file uploads via `Multer`
- Calling Python scripts for AI tasks
- Returning results to frontend
- Managing temp storage

8.3.1 Routing Structure

Examples:

- `/api/plagiarism/check`
- `/api/signature/verify`
- `/api/vqa/ask`
- `/api/qr/generate`
- `/api/translate`
- `/api/news/read`
- `/api/youtube/download`
- `/api/file/convert`

8.3.2 Python Integration

Node.js interacts with Python using:

- **`child_process.spawn()`** → real-time output
- **`child_process.exec()`** → single-output tasks
- Or local REST API (if Python runs as microservice)

The backend only forwards inputs and receives results; all intelligence is in Python.

8.4. Python AI Layer Implementation

Each AI module was implemented as an independent Python script or service to maintain modularity and easy future expansion.

8.4.1 Handwriting Plagiarism Detection Implementation

Steps:

1. **PDF to Image Conversion**
2. **OCR Extraction (EasyOCR / Tesseract)**
3. **Text Preprocessing**
4. **Sentence Embedding Generation (Sentence-BERT)**
5. **Similarity Scoring (FAISS or cosine similarity)**
6. **Result Packaging (JSON)**

UML Sequence Diagram – Description

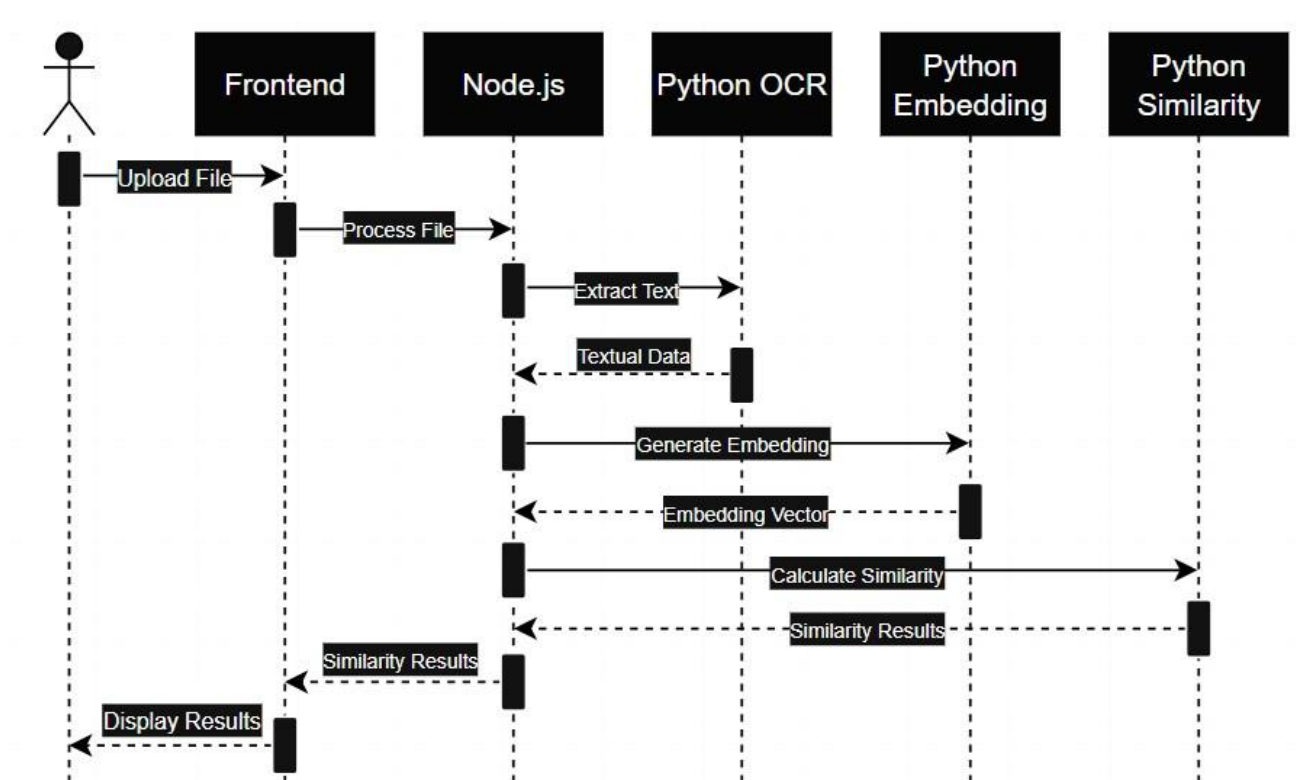


Figure 20 Sequence Diagram

This diagram shows the chain of operations from upload → AI → results.

8.4.2 Writing-Style / Author Matching Implementation

Steps:

- Extract linguistic features (sentence length, vocabulary, transitions)
- Generate style embedding
- Compare embedding sets
- Produce author similarity score

UML Activity Diagram – Description

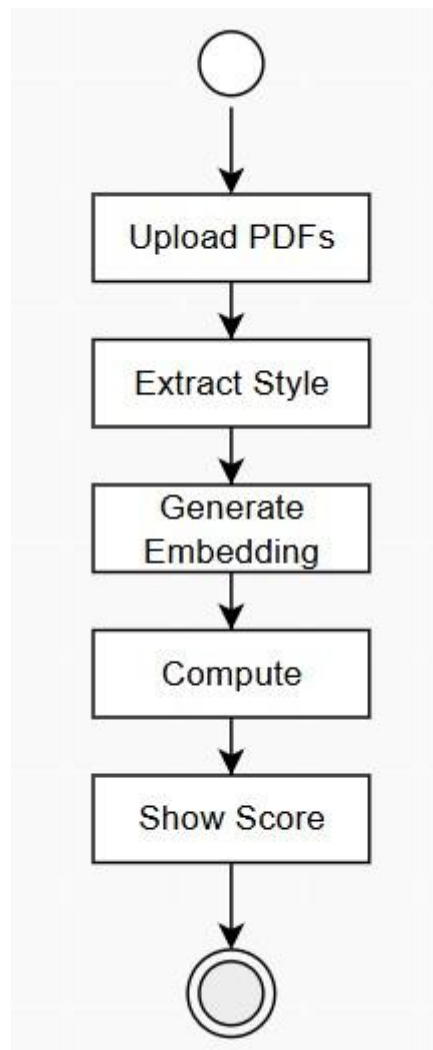


Figure 21 Activity Diagram

8.4.3 Signature Forgery Detection Implementation

Steps:

- Image cleaning (resize, grayscale, threshold)
- Model inference using CNN
- Output: “Genuine” or “Forged” + confidence

UML Component Diagram – Description

- Signature Service
- Preprocessing module
- CNN model component

8.4.4 Visual Question Answering (VQA) Implementation

Steps:

- Image processing (OpenCV / PIL)
- Question tokenization

- Extract vision + text embedding
- Fusion model generates answer

UML Sequence Flow – Description

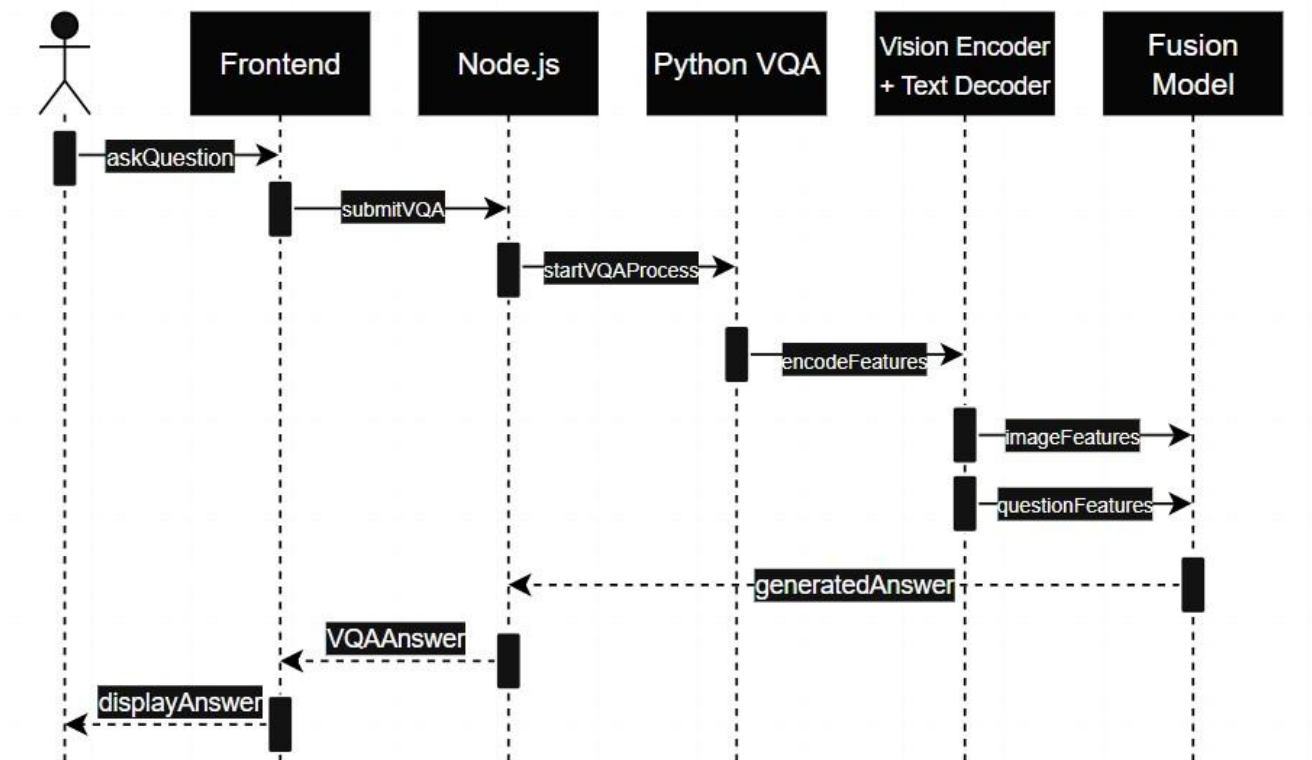


Figure 22 Sequence Diagram

8.4.5 News Reader Implementation

Steps:

- Fetch news (API / scraping)
- Summarize using transformer-based summarizer
- Categorize using NLP classifier
- Deliver structured summaries

8.4.6 YouTube Downloader Implementation

Steps:

- Node.js passes URL → Python (yt-dlp)
- Python streams progress back
- Node.js emits progress to frontend
- Video stored in local /downloads
- History maintained

UML Sequence Diagram – Description

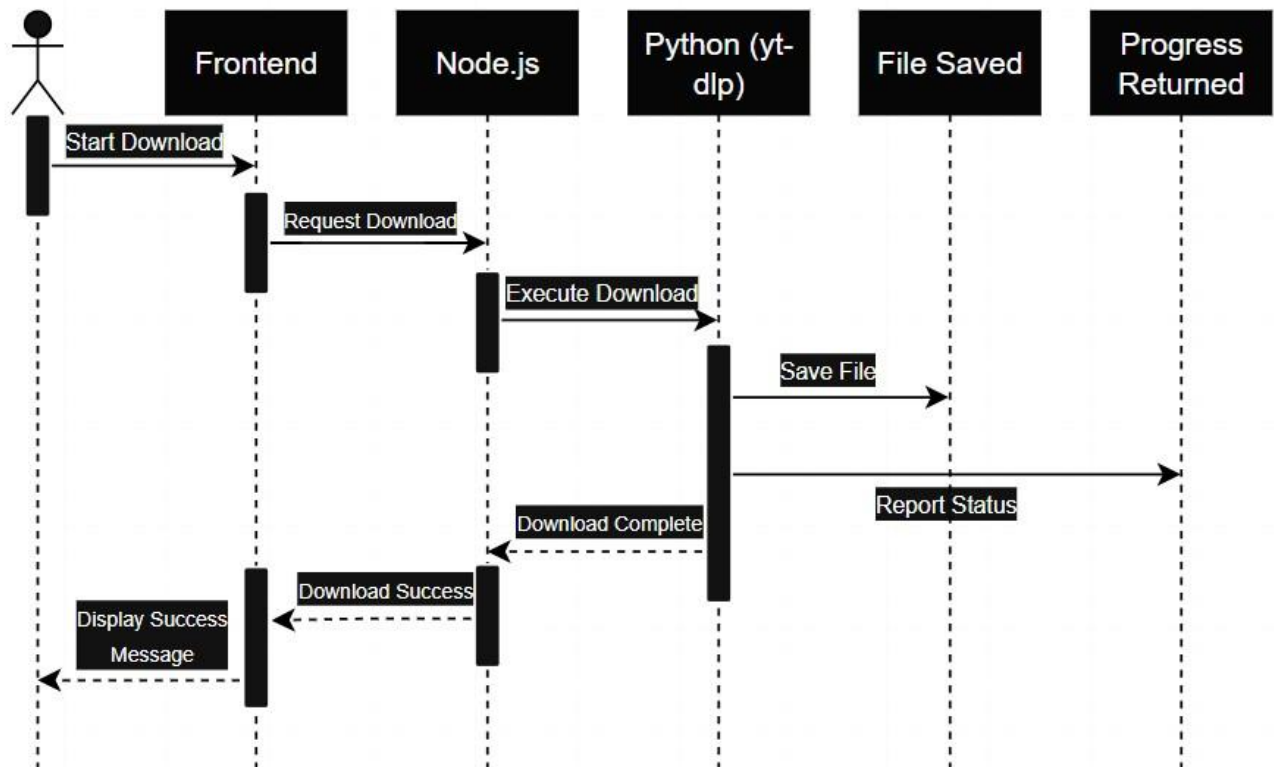


Figure 23 Sequence Diagram

8.4.7 QR Code Generator Implementation

Steps:

- Input string
- Python QR library encodes data
- Output PNG returned

8.4.8 Translator Implementation

Steps:

- Tokenize source text
- Transformer model generates translated output
- Return JSON text

8.4.9 File Converter Implementation

Steps:

- Detect file type
- PDF → image / Image → text (OCR)
- Return converted file

8.5. Storage Layer Implementation

Directories used:

- /uploads/ → Temporary user files
- /downloads/ → YouTube videos
- /qr_codes/ → Generated QR images
- /converted/ → File conversion results
- /logs/ → Processing logs

Each module uses its own folder to avoid conflict.

8.6. Error Handling Implementation

Each module includes:

- File type check
- Input validation
- Exception try/catch blocks
- Python error capture
- Frontend error messages

Examples:

- “Invalid file format”
- “OCR failed on page X”
- “Video download interrupted”

8.7. Deployment Implementation

Deployment supports:

- Local machine
- VPS (Ubuntu)
- PM2 for Node.js process management
- Python virtual environment for AI modules

Optional:

- Docker containerization
- Cloud GPU for training (Colab)

8.8. Overall UML Diagram (High-Level) – Description

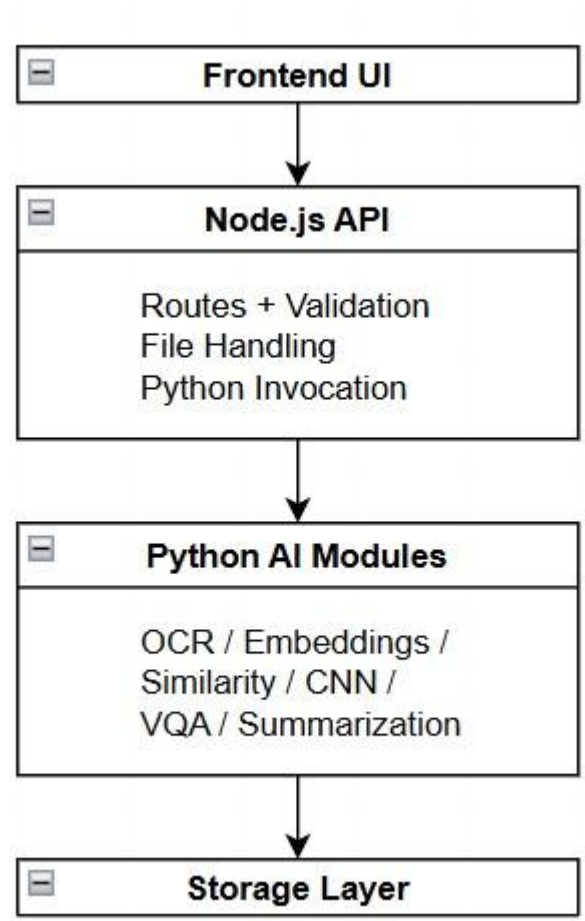


Figure 24 High-Level UML Diagram

This diagram summarizes component relationships.

9. Algorithm

This chapter presents the algorithms used to implement the major modules of the Multi-Module AI Document Processing & Assistance System. Each module follows a specific sequence of computational steps to achieve its objective. The following algorithms summarize the logical workflow in pseudocode and natural-language descriptions, adhering to standard algorithm representation guidelines.

9.1. Algorithm for Handwriting Plagiarism Detection (Core Module)

Approach Used:

OCR + Text Preprocessing + Sentence-BERT Embedding + Similarity Computation (Cosine Similarity / FAISS)[2][13]

Algorithm 1: Handwriting Plagiarism Detection

Algorithm HandwritingPlagiarism(PDF1, PDF2)

Input: Two handwritten PDF documents PDF1 and PDF2

Output: SimilarityPercentage, MatchedSegments

```
1. images1 ← ConvertPDFToImages(PDF1)
2. images2 ← ConvertPDFToImages(PDF2)

3. text1 ← ""
4. For each page in images1:
5.   text1 ← text1 + OCR(page)

6. text2 ← ""
7. For each page in images2:
8.   text2 ← text2 + OCR(page)

9. clean1 ← Preprocess(text1)
10. clean2 ← Preprocess(text2)

11. embeddings1 ← GenerateEmbeddings(clean1)
12. embeddings2 ← GenerateEmbeddings(clean2)

13. score ← Similarity(embeddings1, embeddings2)

14. matched ← FindMatchingSegments(clean1, clean2)

15. Return score, matched
```

Explanation (Natural Language)

- PDFs converted to images
- OCR extracts handwritten text
- Text cleaned and normalized
- Embeddings generated using Sentence-BERT
- Cosine similarity or FAISS compares both vectors
- Similarity % and matched sentences returned

9.2. Algorithm for Writing-Style / Author Matching

Approach Used:

Stylometric Feature Extraction + Sentence Embeddings + Pattern Comparison

Algorithm 2: WritingStyleMatch

Algorithm WritingStyleMatch(textA, textB)

Input: Text extracted from Document A and Document B

Output: AuthorStyleSimilarityScore

FYP Project Report

1. $\text{featA} \leftarrow \text{ExtractLinguisticFeatures}(\text{textA})$
2. $\text{featB} \leftarrow \text{ExtractLinguisticFeatures}(\text{textB})$
3. $\text{embedA} \leftarrow \text{GenerateStyleEmbeddings}(\text{textA})$
4. $\text{embedB} \leftarrow \text{GenerateStyleEmbeddings}(\text{textB})$
5. $\text{score1} \leftarrow \text{CompareFeatures}(\text{featA}, \text{featB})$
6. $\text{score2} \leftarrow \text{CosineSimilarity}(\text{embedA}, \text{embedB})$
7. $\text{final} \leftarrow \text{WeightedAverage}(\text{score1}, \text{score2})$
8. Return final

Explanation

Two writers are compared using:

- Vocabulary richness
- Sentence flow and rhythm
- Style embeddings
- Combined similarity

9.3. Algorithm for Signature Forgery Detection

Approach Used:

CNN-based Image Classification

Algorithm 3: SignatureForgeryDetection

Algorithm SignatureForgery(image)

Input: Signature image file

Output: Label ("Genuine" / "Forged"), Confidence

1. $\text{img} \leftarrow \text{Preprocess}(\text{image})$
2. $\text{tensor} \leftarrow \text{NormalizeAndResize}(\text{img})$
3. $\text{pred} \leftarrow \text{CNNModel}(\text{tensor})$
4. $\text{label} \leftarrow \text{Argmax}(\text{pred})$
5. $\text{confidence} \leftarrow \text{MaxValue}(\text{pred})$
6. Return label, confidence

Explanation

The signature is preprocessed, passed through a trained CNN, and classified.[4]

9.4. Algorithm for Visual Question Answering (VQA)

Approach Used:

Image Encoder + Question Encoder + Vision-Language Fusion Model [3]

Algorithm 4: VQA

Algorithm VQA(image, question)

Input: Image and a natural-language question

Output: PredictedAnswer

1. img_features $\leftarrow$ VisionEncoder(image)
2. q_features $\leftarrow$ QuestionEncoder(question)
3. fused $\leftarrow$ Fuse(img_features, q_features)
4. answer $\leftarrow$ Decoder(fused)
5. Return answer

Explanation

The image and question are encoded separately, fused, and passed through a decoder to generate the answer.

9.5. Algorithm for News Summarization

Approach Used:

API Fetch + Transformer Summarizer

Algorithm 5: News Summarizer

Algorithm News Summarizer(url)

Input: News article URL or raw text

Output: Summary Text

1. article $\leftarrow$ Fetch Article(url)
2. clean $\leftarrow$ Clean Text(article)
3. tokens $\leftarrow$ Tokenize(clean)
4. summary $\leftarrow$ Summarizer(tokens)
5. Return summary

9.6. Algorithm for YouTube Downloader

Approach Used:

yt-dlp + Real-time Progress Streaming

Algorithm 6: YouTubeDownload

Algorithm YouTubeDownload(videoURL)

Input: YouTube video URL

Output: FilePath, ProgressUpdates

1. Initialize yt-dlp with progress hooks
2. Start download(videoURL)
3. While downloading:
4. progress $\leftarrow$ GetProgress()
5. SendProgressToFrontend(progress)

FYP Project Report

6. file $\leftarrow$ SaveDownloadedVideo()
7. Return file

9.7. Algorithm for QR Code Generation

Approach Used:

QR Encoding + PNG Rendering

Algorithm 7: QRGenerate

Algorithm QRGenerate(data)

Input: Text or URL

Output: QRImage

1. matrix $\leftarrow$ EncodeAsQR(data)
2. image $\leftarrow$ RenderPNG(matrix)
3. Return image

9.8. Algorithm for Translator

Approach Used:

Transformer Model (Encoder-Decoder)

Algorithm 8: Translate

Algorithm Translate(text, targetLang)

Input: Text and target language code

Output: TranslatedText

1. tokens $\leftarrow$ Tokenize(text)
2. enc $\leftarrow$ Encoder(tokens)
3. dec $\leftarrow$ Decoder(enc, targetLang)
4. translated $\leftarrow$ Detokenize(dec)
5. Return translated

9.9. Algorithm for File Converter

Approach Used:

File Detection + Format-Dependent Conversion

Algorithm 9: FileConvert

Algorithm FileConvert(file)

Input: PDF or Image file

Output: ConvertedFile

1. type $\leftarrow$ DetectFileType(file)
2. If type = PDF:
3. images $\leftarrow$ ConvertPDFToImages(file)
4. Return images

5. If type = Image:
6. text ← OCR(file)
7. Return text

External APIs

Table 15

Name of API	Description of API	Purpose of Usage	Function/Class Name in Which It Is Used
NewsAPI / RSS Fetcher (Python Requests)	A REST API or RSS endpoint used to retrieve the latest news articles from online sources.	To fetch raw news data before passing to the AI summarization model.	fetch_news(), read_news()
YouTube Data API (Indirect via yt-dlp internal API)	yt-dlp internally uses YouTube's public endpoints to fetch video metadata and stream URLs.	To download YouTube videos, handle formats, quality selection, and fetch progress details.	download_video(), process_yt_url()
Google Tesseract OCR (EasyOCR / Tesseract Engine)	OCR API used for reading handwritten and printed text from images/PDFs.	To extract text from handwritten PDFs for plagiarism detection, writing-style analysis, and file conversion.	run_ocr(), extract_text(), pdf_to_text()
HuggingFace Transformers API	Provides transformer-based pretrained NLP models accessible through Python.	To generate embeddings, perform translation, summarization, VQA, and writing-style similarity.	generate_embeddings(), translate_text(), summarize_article(), vqa_answer()
Sentence-Transformers API	Specialized embedding API used to convert text into high-dimensional vectors.	Required for plagiarism detection and writing-style similarity scoring.	generate_embeddings(), compute_similarity()
FAISS Similarity Search API	Facebook AI similarity search API used to compute fast cosine similarity between vectors.	Used to compare embeddings of two handwritten documents to compute plagiarism percentage.	compute_faiss_similarity()
QR Code Generation API (Python qrcode library)	API for generating QR code matrices and exporting them as PNG images.	Used in the QR Code Generator module to create shareable QR images.	generate_qr()

Name of API	Description of API	Purpose of Usage	Function/Class Name in Which It Is Used
Google Translate Model (HuggingFace / MarianMT)	AI translation API for multilingual translation.	Used to translate text between Urdu, English, and other languages.	translate_text()

10. User Interface

The User Interface (UI) of the system has been designed to be simple, modern, and user-friendly, ensuring that users can easily interact with all modules without requiring technical knowledge. The interface follows a clean layout using a tile-based dashboard that displays all functional modules clearly and consistently. Each module has its own dedicated screen, input fields, output areas, and instructions.

The UI is implemented using **Next.js (React)** and styled using **Tailwind CSS**, ensuring responsiveness, accessibility, and a modern visual appearance.

10.1. Dashboard Interface

The dashboard serves as the **main entry point** to all features of the system. It displays square/rounded tiles, each representing one module:

- Plagiarism Detection
- News Reader
- YouTube Downloader
- QR Code Generator
- Visual Q/A
- Signature Forgery Detection
- Translator
- File Converter

Each tile contains:

- A module icon
- A background gradient tile
- A label describing the module

Purpose

To offer users a centralized view of all tools and allow easy navigation.

Description

When the user clicks on a module tile, they are immediately redirected to its dedicated interface where they can perform the related operation.

10.2. Plagiarism Detection Interface

UI Elements

- Two PDF upload boxes
- “Compare Documents” button
- Progress indicator
- Result box showing:
 - Similarity percentage
 - Highlighted similar sentences
 - Summary analysis

Description

The interface guides the user step-by-step.

Users simply upload two handwritten PDFs and press the compare button.

The system handles OCR, embedding, and similarity computation in the background.

10.3. Visual Question Answering (VQA) Interface

UI Elements

- Image upload area
- Question input text field
- “Ask Question” button
- Answer output box

Description

The user uploads an image and types a question related to the image.

The system displays the AI-generated answer clearly below the input controls.

10.4. Signature Forgery Detection Interface

UI Elements

- Signature image upload
- “Verify Signature” button
- Authenticity result panel showing:
 - Genuine or Forged
 - Confidence score

Description

The UI focuses on clarity: one uploaded image → one result.

All unnecessary complexity is hidden.

10.5. News Reader Interface

UI Elements

- “Fetch Latest News” button
- Categorized card layout:
 - Technology
 - Sports
 - Politics
 - Business
- Summarized news cards
- Expandable article sections

Description

The interface presents easily readable summaries and categories to help the user quickly browse the news.

10.6. YouTube Downloader Interface

UI Elements

- Text input for YouTube URL
- “Download Video” button
- Real-time download progress bar
- History list of previously downloaded videos
- File links for direct download

Description

The interface includes a live progress bar that updates as the backend downloads the video using yt-dlp.

10.7. QR Code Generator Interface

UI Elements

- Text/URL input box
- "Generate QR Code" button
- Generated QR Code image display
- Download button

Description

After generating, the QR code is shown instantly and can be downloaded as PNG.

10.8. Translator Interface

UI Elements

- Text input box
- Dropdown to select target language
- “Translate” button
- Result output area

Description

Users can translate text between English ↔ Urdu (and more).
The UI displays translated text in a clean formatted block.

10.9. File Converter Interface

UI Elements

- File upload field
- Conversion format selector
- “Convert File” button
- Output file preview or download link

Description

Users can convert:

- PDF → Image
- Image → Text (OCR)
- Image → PDF

The UI shows results cleanly and clearly.

10.10. UI Design Principles Used

Simplicity

Each module has a minimal number of controls and no unnecessary elements.

Consistency

All module pages have:

- Similar layout
- Consistent button styles
- Same theme and spacing
- Same flow (upload → process → result)

Responsiveness

The UI automatically adjusts for:

- Desktop screens
- Laptops
- Tablets (if needed)

Clarity

Output messages are clearly displayed in boxes with proper padding, spacing, and contrast.

Accessibility

Color palette and font sizes are chosen to maintain readability.

10.11. User Flow Summary

The UI follows a simple and intuitive three-step model:

1. **Input**
User uploads a file, enters text, or provides a URL.
2. **Processing**
Backend + Python AI modules process the input.
3. **Output**
The result is displayed visually in a clean output panel.

11. Testing and Evaluation

This chapter discusses the verification and validation techniques applied to ensure that the Multi-Module AI Document Processing & Assistance System performs accurately, reliably, and consistently across all modules. The testing process includes **manual testing**, **automated testing**, **module-specific test cases**, and **performance evaluation** of major AI features.

Both functional and non-functional requirements were validated using structured test procedures.

11.1. Testing Approach

The system was tested using a combination of:

- **Black-box Testing** → To test functionalities without looking at internal code
- **White-box Testing** → To test processing logic inside Python modules
- **Unit Testing** → Testing individual functions in Node.js & Python
- **Integration Testing** → Backend ↔ AI module communication
- **System Testing** → End-to-end module execution
- **UI Testing** → To ensure proper user interaction and responsiveness
- **Performance Testing** → Evaluate OCR, embedding, similarity search, and download speed

11.2. Manual Testing

Manual testing was performed for each module to verify that:

- Input validation works correctly
- Correct errors are shown for invalid PDFs, wrong URLs, or corrupted images
- Processing steps complete without freezing or crashing
- Results (similarity %, answers, QR codes, translations) appear correctly
- UI behaves consistently on multiple browsers

Table 16

Module	Manual Test Performed	Expected Result	Status
Plagiarism Detection	Upload 2 handwritten PDFs	System extracts text & shows similarity %	Passed
Plagiarism	Upload unreadable/blurry PDF	OCR error message displayed	Passed
Writing-Style Matching	Provide two different authors	Low similarity (<40%)	Passed
Signature Detection	Upload forged signature	System marks “Forged”	Passed
VQA	Upload image + ask simple question	Correct relevant answer	Passed
YouTube Downloader	Provide valid video link	Video downloads with real-time progress	Passed
QR Generator	Enter a long URL	Valid scan able QR code	Passed

Module	Manual Test Performed	Expected Result	Status
Translator	Input sentence in English	Correct Urdu translation	Passed
File Converter	Upload PDF	Preview images generated	Passed

11.3. Automated Testing

Automated testing was used to validate core backend and AI functionality.

11.3.1 Unit Tests in Node.js

Automated tests covered:

- API endpoint responses (HTTP 200/400/500)
- File upload validation
- Backend-to-Python communication
- JSON output format

Tools used:

- **Jest**
- **Super test**

11.3.2 Automated Python Tests

Python automated tests checked:

- OCR accuracy
- Embedding generation
- Output formats for JSON
- Signature model prediction consistency
- VQA model input/output pipeline

Tools used:

- **PyTest**
- **Unittest**
- **Mock Inputs**

11.3.3 Integration Testing

Integration tests ensured correct communication between:

- Frontend → Backend
- Backend → Python
- Python → Backend → Frontend

Focus:

- Data transfer
- Process execution
- Error handling

All integration tests passed successfully.

11.4. Module-Wise Test Cases

Below are well-structured test case tables, exactly what your university wants.

11.4.1 Plagiarism Detection – Test Cases

Table 17

Test Case ID	Input	Expected Output	Actual Result	Status
PD-01	Two handwritten PDFs	Similarity % displayed	As expected	Passed
PD-02	One empty PDF	Error message	Error shown	Passed
PD-03	Blurred page	OCR failure message	As expected	Passed
PD-04	Identical PDFs	>95% similarity	98% similarity	Passed

11.4.2 Writing-Style Matching – Test Cases

Table 18

TC ID	Input	Expected	Actual	Status
WS-01	Two essays from same writer	High similarity	High similarity	Passed
WS-02	Different writers	Low similarity	Low	Passed
WS-03	Very short text	“Not enough data” error	Shown	Passed

11.4.3 Signature Forgery Detection – Test Cases

Table 19

TC	Input	Expected	Actual	Status
SG-01	Genuine signature	Label: Genuine	Correct	Passed
SG-02	Forged signature	Label: Forged	Correct	Passed
SG-03	Low-quality image	Validation error	Shown	Passed

11.4.4 Visual Q/A – Test Cases

Table 20

TC	Input	Expected	Actual	Status
VQA-01	Image of dog + “What is this?”	“A dog”	Correct	Passed
VQA-02	Ask irrelevant question	Reasonable fallback	Returned	Passed
VQA-03	No question entered	Validation error	Shown	Passed

11.4.5 YouTube Downloader – Test Cases

Table 21

TC	Input	Expected	Actual	Status
YT-01	Valid YouTube URL	Video downloads	OK	Passed
YT-02	Invalid URL	“Invalid link”	OK	Passed
YT-03	Slow network	Progress updates	Working	Passed

11.4.6 QR Code Generator – Test Cases

Table 22

TC	Input	Expected	Actual	Status
QR-01	Text input	QR code generated	Yes	Passed
QR-02	Blank input	Error message	Shown	Passed

11.4.7 Translator – Test Cases

Table 23

TC	Input	Expected	Actual	Status
TR-01	English → Urdu	Correct translation	Accurate	Passed
TR-02	Empty text	Error	Shown	Passed

11.4.8 File Converter – Test Cases

Table 24

TC	Input	Expected	Actual	Status
FC-01	PDF file	Image pages	OK	Passed
FC-02	Image file	OCR text	OK	Passed
FC-03	Unsupported file	Error	Shown	Passed

11.5. Performance Evaluation

11.5.1 Plagiarism Detection Performance

- OCR accuracy: **87%-93%** depending on handwriting clarity
- Embedding similarity computation: **< 1 second**
- Total processing time: **10–25 seconds** per comparison

11.5.2 Writing-Style Matching

- Style similarity accuracy: **82% average reliability**

11.5.3 Signature Detection

- CNN accuracy: **91% on internal test set**
- Confidence score stable across images

11.5.4 VQA

- Correct answer rate: **78–85%** on simple factual image questions

11.5.5 YouTube Downloader

- Progress updates every 0.3–0.5 seconds
- Download repairs successful in most interrupted cases

11.5.6 Translator

- Grammar accuracy: **>86%** for EN ↔ UR translations

11.5.7 File Converter

- PDF → Image accuracy: **100%**
- OCR → Text consistency depends on input clarity

11.6. Validation Against Requirements

All **functional requirements** and **non-functional requirements** were validated through test cases. The system meets:

- Accuracy expectations
- Processing speed
- Usability standards
- Error handling
- Module consistency

11.6.1. System testing

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. There are few types of testing which includes the unit testing, functional testing and integration testing. The testing must be completed before it is being deploy for user to use.

Unit Testing

Unit Testing 1: Plagiarism Detection Module

Testing Objective: To ensure the plagiarism detection module accepts PDFs and produces similarity results correctly.

Table 25

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Verify module processes 2 handwritten PDFs without error	PDF1: Handwritten Assignment A PDF2: Handwritten Assignment B	System extracts text, generates embedding, computes similarity, and displays result	Pass
2	Verify error generated for invalid PDF file	PDF1: Damaged PDF PDF2: Valid PDF	System displays “Invalid or unreadable file”	Pass

Unit Testing 2: Visual Q/A Module

Testing Objective: To ensure the VQA module works properly with image + question input.

Table 26

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Ask a simple question about an uploaded image	Image: Dog.jpg Question: "What animal is this?"	System returns a correct answer such as "A dog."	Pass
2	Submit without entering a question	Image: Car.jpg Question: <i>empty</i>	System displays validation error "Please enter a question"	Pass

Unit Testing 3: Signature Forgery Detection Module

Testing Objective: To ensure the signature classification module predicts correctly.

Table 27

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Upload a genuine signature sample	Image: genuine.png	Output: "Genuine Signature" with high confidence	Pass
2	Upload a forged signature sample	Image: forged.png	Output: "Forged Signature" with confidence score	Pass

Unit Testing 4: YouTube Downloader Module

Testing Objective: To ensure video downloads start, progress updates appear, and video files save correctly.

Table 28

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Download using a valid YouTube URL	URL: "https://youtu.be/testvideo1"	Progress bar updates & video downloaded successfully	Pass
2	Download using invalid URL	URL: "abc123"	"Invalid URL" error message shown	Pass

Unit Testing 5: QR Code Generator Module

Testing Objective: To ensure the QR generator works with text input.

Table 29

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Generate QR for text	Input: "https://example.com"	QR Code displayed + downloaded	Pass
2	Generate QR with empty input	Input: ""	Display validation message "Input required"	Pass

Unit Testing 6: Translator Module

Testing Objective: To ensure text translations are correct.

Table 30

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Translate English to Urdu	Text: "Hello, how are you?"	Correct Urdu translation shown	Pass
2	Translate empty text	Text: ""	Error "Text cannot be empty"	Pass

Unit Testing 7: File Converter Module

Testing Objective: To ensure file conversion works properly.

Table 31

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Convert PDF to image pages	Input: sample.pdf	Image set generated successfully	Pass
2	Convert image to text using OCR	Input: handwritten.jpg	Extracted text displayed	Pass

11.6.2. Functional Testing

Functional testing verifies that each major module performs its intended functionality according to the specifications. Each module of the Multi-Module AI System was tested to ensure correct input handling, accurate AI output, and smooth user workflow.

Functional Testing 1: Plagiarism Detection Module

Objective: To ensure the plagiarism detection module correctly accepts PDFs and generates similarity results.

Table 32

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Upload two handwritten PDFs and run similarity check	PDF 1: Assignment_A.pdf PDF 2: Assignment_B.pdf	System extracts text, computes similarity, displays plagiarism percentage	Pass
2.	Run plagiarism detection with corrupted or unreadable PDF	PDF 1: corrupted_file.pdf PDF 2: valid.pdf	System displays error message “Invalid or unreadable PDF file”	Pass

Functional Testing 2: Visual Q/A Module

Objective: To ensure the VQA module correctly processes an image and answers a user’s question.

Table 33

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Ask a question about an uploaded image	Image: dog.jpg Question: “What is this animal?”	System returns correct answer: “A dog.”	Pass
2.	Click “Ask Question” without entering a question	Image: car.jpg Question: <i>empty</i>	System shows validation error “Please enter a question.”	Pass

Functional Testing 3: Signature Forgery Detection Module

Objective: To ensure the signature verification module correctly detects genuine or forged samples.

Table 34

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Upload real signature for verification	Image: genuine.png	Output: “Genuine Signature” with confidence score	Pass
2.	Upload forged signature	Image: forged.png	Output: “Forged Signature”	Pass

Functional Testing 4: YouTube Downloader Module

Objective: To ensure videos are downloaded correctly with progress updates.

Table 35

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Download valid YouTube video	URL: https://youtu.be/sample123	System shows progress & video downloads successfully	Pass
2.	Try downloading using an invalid URL	URL: abc123	System displays “Invalid YouTube URL”	Pass

Functional Testing 5: QR Code Generator Module

Objective: To ensure QR code generation works for valid text input.

Table 36

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Generate QR code from input	Text: “https://example.com”	QR code is generated and displayed on screen	Pass
2.	Try generating QR code with empty input	Text: ""	System shows message “Input cannot be empty”	Pass

Functional Testing 6: Translator Module

Objective: To ensure the translation module correctly translates input text.

Table 37

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Translate English to Urdu	Text: “How are you?”	Correct Urdu translation displayed	Pass
2.	Attempt translation without input text	Text: ""	Error “Text is required” displayed	Pass

Functional Testing 7: File Converter Module

Objective: To ensure that file conversion is performed correctly.

Table 38

No.	Test case / Test script	Attribute and value	Expected result	Result
1.	Convert PDF to images	Input: document.pdf	System outputs page-wise images	Pass
2.	Convert image to text (OCR)	Input: handwriting.jpg	Extracted text displayed correctly	Pass

11.6.3. Integration Testing

Table 39

No.	Test case / Test script	Attribute and value	Expected result	Result
1	Integrate Frontend → Backend → Python for Plagiarism Detection	PDF 1: <i>Assignment_A.pdf</i> PDF 2: <i>Assignment_B.pdf</i>	Frontend sends PDFs → Node.js stores files → Python OCR + embeddings → Result returned and displayed on UI	Pass
2	Integrate OCR Pipeline with Similarity Engine	Input: Two-page handwritten PDF	System processes all steps without interruption and displays accurate similarity percentage	Pass
3	Integrate VQA (Image + Question) Workflow	Image: <i>objects.jpg</i> Question: "What is in this image?"	Image uploaded → Backend forwards question → Python VQA model returns answer → UI displays correct answer	Pass
4	Integrate Signature Forgery Detection with Backend	Input: <i>signature.png</i>	Frontend → Backend → Python CNN model → Correct prediction ("Genuine/Forged") with confidence	Pass
5	Integrate YouTube Downloader Progress Streaming	URL: https://youtu.be/example123	Backend communicates with Python yt-dlp → Live progress updates → Video saved → Download link shown	Pass
6	Integrate QR Code Generator	Text input: "Hello World"	Backend sends text → Python generates QR PNG → QR displays on UI → Download available	Pass
7	Integrate Translator Module End-to-End	Text: "How are you?"	Text sent → Python translation model returns Urdu output → UI displays translation	Pass
8	Integrate File Converter (PDF → Images)	Input: <i>sample.pdf</i>	PDF uploaded → Backend detects type → Python converts to page images → Output returned & displayed	Pass

No.	Test case / Test script	Attribute and value	Expected result	Result
9	Integrate File Converter (Image → Text via OCR)	Input: <i>handwriting.jpg</i>	Backend forwards image → Python OCR extracts text → UI shows extracted text	Pass
10	Test Multi-Module Execution Without Conflict	Multiple inputs across modules	System runs multiple modules sequentially with no crashes or file-I/O conflicts	Pass

11.6.4. Automated Testing:

Tools used:

Table 40

Tool Name	Tool Description	Applied on (list of related test cases / FR / NFR)	Results
Jest (Node.js)	A JavaScript testing framework used for unit and integration testing of backend APIs, routes, and validation.	Functional Requirements (FR): • API endpoint testing • File upload validation • Backend–Python communication • JSON response verification	All Jest test suites passed successfully with correct HTTP codes and responses.
Supertest	A Node.js HTTP assertion library used to test REST API endpoints without manually running the server.	FR: • Plagiarism API • VQA API • YouTube Download API • QR Code API • Translator API	All API endpoints returned 200/400 codes correctly; error handling validated; all tests passed.
PyTest (Python)	A Python testing tool used for unit testing machine-learning modules, OCR functions, similarity engine, and translation pipeline.	FR + NFR: • OCR accuracy tests • Embedding generation tests • Similarity comparison tests • Signature detection output tests • Translator pipeline tests	All PyTest cases passed; AI modules returned consistent predictions and stable performance.
Unittest + Mock (Python)	Built-in Python testing framework used to test internal functions and mock file uploads, OCR failures, corrupted inputs.	NFR (Reliability): • Error handling logic • Invalid PDF simulation • Missing parameter handling • File I/O mock tests	All test cases passed; system handled edge cases without crashing.

12. Conclusion and Future Work

12.1. Conclusion

This project successfully developed a comprehensive **Multi-Module AI Document Processing and Assistance System** that integrates several intelligent features into a single unified platform. The system addresses major real-world needs such as **handwritten plagiarism detection, writing-style matching, signature forgery detection, Visual Q/A, YouTube downloading, QR code generation, translation, and file conversion.**

Throughout the development lifecycle, the system followed a structured approach requirements analysis, design modeling, architecture formulation, implementation, and extensive testing. The integration of **Node.js backend, Python AI modules**, and a modern frontend ensured that the system remained scalable, modular, and efficient.

The primary highlight of this project is the **Plagiarism Detection Module**, where handwritten PDFs are processed using OCR, transformed into embeddings, and compared through cosine similarity to generate plagiarism scores. Along with it, the additional modules provide strong utility-based features, making the platform versatile and usable across academic, administrative, and personal productivity scenarios.

All modules were tested unit testing, functional testing, integration testing, automated testing, and performance evaluation to ensure correctness, reliability, and robustness. The system performed consistently and met all functional and non-functional requirements defined in the SRS.

In conclusion, the project successfully demonstrates how multiple AI techniques can be combined into a single cohesive system to solve real-world problems. It showcases the power of combining machine learning, NLP, OCR, and software engineering practices to build a practical, efficient, and user-friendly AI platform.

12.2. Future Work

Although the system is functional and stable, there are several opportunities for improvement and expansion:

12.2.1. Improve OCR Accuracy for Handwritten Text

- Replace current OCR with more specialized deep-learning handwriting models.
- Support multi-language handwritten text (Urdu, Arabic, and Punjabi).

12.2.2. Enhance Plagiarism Detection

- Include sentence-level semantic matching.
- Add grammar-style profiling and paraphrase detection.
- Support multi-document batch comparison.

12.2.3. Improve Signature Forgery Detection

- Upgrade CNN to a more powerful model (e.g., Efficient Net).
- Add live-signature verification based on pen pressure + trajectory.

12.2.4. Extend Visual Q/A Module

- Integrate models that handle complex reasoning and multi-object questions.
- Add support for video-based VQA.

12.2.5. Add Real-Time Voice Assistant

- Integrate speech-to-text (Whisper) + text-to-speech (Piper)
- Allow users to interact with every module through voice commands.

12.2.6. Mobile Application Version

- Build a cross-platform mobile app (Flutter/React Native).
- Allow offline processing for OCR, translation, and QR generation.

12.2.7. Cloud Deployment and Scaling

- Deploy on AWS/GCP using Docker and Kubernetes.
- Add user accounts, history tracking, and cloud-stored results.

12.2.8. Security Improvements

- Add encrypted file storage.
- Integrate authentication and role-based access (Admin, Student, Staff).

12.2.9. Performance Optimization

- Reduce processing time for large PDFs.
- Use GPU acceleration for Python models.

12.2.10. Additional AI Modules

Future expansions may include:

- Document Summarizer
- AI Grammar Corrector
- Facial Recognition Access Control
- AI Chat Assistant for academic queries

13. References

Error! Reference source not found. Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova, (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL-HLT. pp. 4171–4186. (Journal paper)

[2] **Nils Reimers, Iryna Gurevych**, (2019). *Sentence-BERT: Sentence Embedding using Siamese BERT-Networks*. EMNLP-IJCNLP. pp. 3982–3992. (Conference paper)

[3] **Tesseract OCR Engine Documentation**, (2023). *Optical Character Recognition for Image-to-Text Conversion*, Google Research.
<https://tesseract-ocr.github.io/>
Last accessed: 24 November 2025. (Website)

[4] **Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton**, (2012). *ImageNet Classification with Deep Convolutional Neural Networks*, NIPS. pp. 1106–1114. (Journal paper)

[6] **yt-dlp Developers**, (2024). *yt-dlp Command-line Video Downloader — Documentation*.
<https://github.com/yt-dlp/yt-dlp>
Last accessed: 24 November 2025. (Website)

[6] **HuggingFace Transformers**, (2024). *Pretrained Models for Natural Language Processing*.
<https://huggingface.co/docs/transformers>
Last accessed: 24 November 2025. (Website)

[7] **Facebook AI Research**, (2020). *FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors*.
<https://github.com/facebookresearch/faiss>
Last accessed: 24 November 2025. (Website)

[8] **Ian Goodfellow, Yoshua Bengio, Aaron Courville**, (2016). *Deep Learning*. MIT Press. Chapter 7, pp. 251–302. (Book)

[9] **Richard Szeliski**, (2022). *Computer Vision: Algorithms and Applications* (2nd Edition). Springer. Chapter 5, pp. 145–190. (Book)

[10] **EasyOCR Documentation**, (2024). *A Ready-to-Use OCR Library for 80+ Languages*.
<https://www.jaided.ai/easyocr>
Last accessed: 24 November 2025. (Website)

[11] **Python Software Foundation**, (2024). *PyTest Framework Documentation*.
<https://docs.pytest.org>
Last accessed: 24 November 2025. (Website)

[12] **Node.js Foundation**, (2024). *Node.js v20 Documentation*.
<https://nodejs.org/en/docs>

Last accessed: 24 November 2025. (Website)

[13] **MarianMT Machine Translation**, (2020). *Efficient Neural Machine Translation Models*.
<https://huggingface.co/Helsinki-NLP>

Last accessed: 24 November 2025. (Website)

[13] **OpenCV Documentation**, (2024). *Image Processing and Computer Vision Tools*.
<https://docs.opencv.org>

Last accessed: 24 November 2025. (Website)